Assignment #6: jQuery (Selectors, Effects, Animations)

Comprehensive Technical Implementation, Verification & Academic Report

Student Name	Azamat Bukharbayev
Academic Group	IT-2510 (Astana IT University)
Selected Task Options	Task 1: Selector Detective | Task 2: The Party in Motion
GitHub Repository	https://github.com/azamatbukharbayev/Assignment6
Course & Semester	Web Technologies 1 (Front End) — Fall 2026

1. Project Overview & Technical Setup

Assignment #6 explores the capabilities of jQuery for advanced DOM selection, structural traversal, animated effects, and interactive interface engineering. The project is situated around a scenario where AITU students gathered at a birthday party in a multi-story house between 19:30 and 22:30. At 21:30, the birthday cake mysteriously disappeared from the kitchen. The assignment is divided into two distinct, independently functioning web applications:

- • Task 1 (Selector Detective): A data-driven website reconstructing the house, floors, and 21 party guests from the authoritative reference illustration. It implements 20 interactive detective case buttons that execute explicit jQuery selector and traversal chains against the rendered DOM to solve queries, animate found cards with a one-time vertical jump, and dynamically report findings.
- • Task 2 (The Party in Motion): An independent animated version extending the party interface with five core effects: (1) Spotlight and Clear via fadeTo(), (2) Expanding Guest Detail Cards growing outward from avatars with descriptive sentences, (3) Cross-house flying guest room transfers with gap animation, (4) Positional chronological sorting using a jQuery FLIP technique, and (5) Strict animation queue management handling rapid interactions.

1.1. Strict Technical Constraints & Compliance

To satisfy the core academic requirements of the university specification, strict engineering boundaries were enforced across both implementations:

- **1.** Exclusive Library Policy: jQuery v3.7.1 is the sole JavaScript library used. No UI frameworks (Bootstrap, Tailwind JS), component libraries (React, Vue), or external animation libraries (GSAP, Anime.js) were introduced.
- **2.** No Inline Event Handlers: No inline 'onclick' attributes exist in HTML; all interactions are managed cleanly via jQuery event listeners (.on('click'), .on('keydown')).
- **3.** Absolute Ban on CSS Transitions and Animations: Neither task1.css nor task2.css contains any 'transition' declarations, '@keyframes' blocks, or CSS animation properties. All dynamic movement, jumping, opacity changes, and resizing are governed exclusively by jQuery effects (animate(), fadeIn(), fadeOut(), fadeTo()).

- **4.** DOM-Based Querying Integrity: Selector detective cases evaluate against the live rendered HTML document tree using DOM classes and data-* attributes. The JavaScript array is strictly used for initial rendering and state synchronization, never as a shortcut substitute for DOM querying.

1.2. Project Architecture & File Organization

The project maintains clean modularity and separation of concerns. The production workspace is organized as follows:

```
Assignment6/
├── index.html                    # Project landing hub & documentation overview
├── task1.html                    # Task 1: Selector Detective web application
├── task2.html                    # Task 2: The Party in Motion web application
├── css/
│   ├── task1.css                 # Task 1 static styles (zero transitions/
animations)
│   └── task2.css                 # Task 2 static styles & modal overlays
├── js/
│   ├── jquery.min.js             # Standalone local jQuery 3.7.1 dependency
│   ├── task1.js                  # Task 1 data model, DOM renderer, 20 selector
cases
│   └── task2.js                  # Task 2 independent mutable model, effects,
sorting
├── test_runner.js                # Headless verification suite (Node.js + jsdom)
├── generate_report.py            # Automated academic report generator
├── README.md                     # Operational manual, test guide, defense notes
├── Assignment6_AzamatBukharbayev.docx # Editable academic report
└── Assignment6_AzamatBukharbayev.pdf  # Final rendered PDF report
```

2. Reconstruction of the Original Party Data

The starting point of the assignment is the precise transcription of the reference illustration ('The Party, 19:30 - 22:30'). Every visible detail in the image—including the house's physical architecture, the vertical floor distribution, the horizontal left-to-right order of guests, guest names, clothing ring colors, arrival timestamps, AITU student affiliations, and every associated item—was transcribed into a structured JavaScript model without omission or modification.

2.1. House Geometry & Floor Distribution

The house consists of two distinct floors and four rooms:

- • Floor 2 (Upper Floor): Contains Balcony (left, 4 guests) and Hall (right, 6 guests).
- • Floor 1 (Ground Floor): Contains Kitchen (left, 6 guests) and Living Room (right, 5 guests).

The initial party comprises exactly 21 guests across 4 rooms.

2.2. Authoritative Initial Party Dataset

Table 1 outlines the complete reconstructed party dataset used as the immutable baseline in Task 1 and initial state in Task 2:

Room	Floor	Name	Clothing Ring	Arrival	AITU?	Items Associated	Init. Order

Balcony	2	**Aigerim**	Green	20:45	Yes	Cup, Cake crumbs	0
Balcony	2	**Aliya**	Black	21:15	No	Phone, Cup	1
Balcony	2	**Madina**	Green	22:10	No	Phone, Bag, Cake crumbs	2
Balcony	2	**Ivan**	Blue	20:10	Yes	Cup	3
Hall	2	**Arman**	Yellow	20:35	No	Bag, Glasses	0
Hall	2	**Sophia**	Purple	21:20	Yes	Phone	1
Hall	2	**Dias**	Green	19:45	Yes	None (empty)	2
Hall	2	**Bekzat**	Purple	20:55	No	Bag, Cup, Cake crumbs	3
Hall	2	**Aruzhan**	Blue	21:50	Yes	Phone	4
Hall	2	**Saule**	Green	21:25	No	Glasses	5
Kitchen	1	**Nurlan**	Red	19:40	Yes	Phone, Cup	0
Kitchen	1	**Emma**	Black	20:15	Yes	Bag	1
Kitchen	1	**Timur**	Green	21:05	Yes	Phone, Bag, Cake crumbs	2
Kitchen	1	**Sergey**	Yellow	20:50	No	None (empty)	3
Kitchen	1	**Yerlan**	Red	21:40	Yes	Glasses, Phone	4
Kitchen	1	**Kamila**	Green	20:05	No	Cup	5
Living room	1	**Daniel**	Blue	19:55	Yes	Glasses	0
Living room	1	**Zarina**	Green	21:10	No	Bag, Cup, Cake crumbs	1
Living room	1	**Natasha**	Yellow	20:30	Yes	None (empty)	2
Living room	1	**Askar**	Blue	21:00	Yes	Phone	3
Living room	1	**Inkar**	Purple	20:20	No	Bag	4

2.3. DOM Construction & Avatar Integration

The party house is assembled dynamically using jQuery DOM construction (createGuestCard). For each guest, an interactive card element (.guest-card) is created containing:

- **1.** Avatar Generation: SVGs sourced from DiceBear Open Peeps (https://api.dicebear.com/9.x/open-peeps/svg?seed=GuestName) encoded safely via encodeURIComponent. If an image fails to load, a resilient inline fallback avatar is substituted.
- **2.** Clothing Outline Ring: A 3.5px solid circular border wrapping the avatar with specific color classes (.ring-red, .ring-blue, .ring-green, .ring-yellow, .ring-black, .ring-purple).

- **3.** AITU Student Badge: A graduation cap badge (🎓) positioned at the top-right of the avatar ring for students having aituStudent: true.
- **4.** Arrival Timestamp: Displayed with tabular numbers alongside a clock icon (⏱ HH:MM).
- **5.** Item Representation: Specific CSS classes (.item-phone, .item-glasses, .item-bag, .item-cup, .item-crumbs) and lightweight accessible text icons with tooltip descriptions. Guests with no items receive class .has-no-items.
- **6.** Data Attributes: Attributes data-id, data-name, data-color, data-arrival, data-aitu, data-room, and data-floor are populated, enabling compound DOM queries.

3. Task 1 — Selector Detective (All 20 Cases)

In Task 1, all 20 detective cases were implemented using jQuery selector chains and traversal methods executed against the rendered HTML DOM. Before running any case (1–17, 20), the house is freshly reconstructed from the original data array to ensure complete test isolation. Cases 18 and 19 represent dynamic mutations, adding Miras via fadeIn() and removing Natasha via fadeOut().

Case 1 — Who is in the hall?

Uses an ID descendant selector to select all guest cards located within the room element whose id is 'room-hall'. Matches all 6 residents currently standing in the Hall.

Code Snippet: Actual jQuery Selector Chain:

```
$("#room-hall .guest-card")
```

Verified Result: Count: 6 guests | Identified: Arman, Sophia, Dias, Bekzat, Aruzhan, Saule

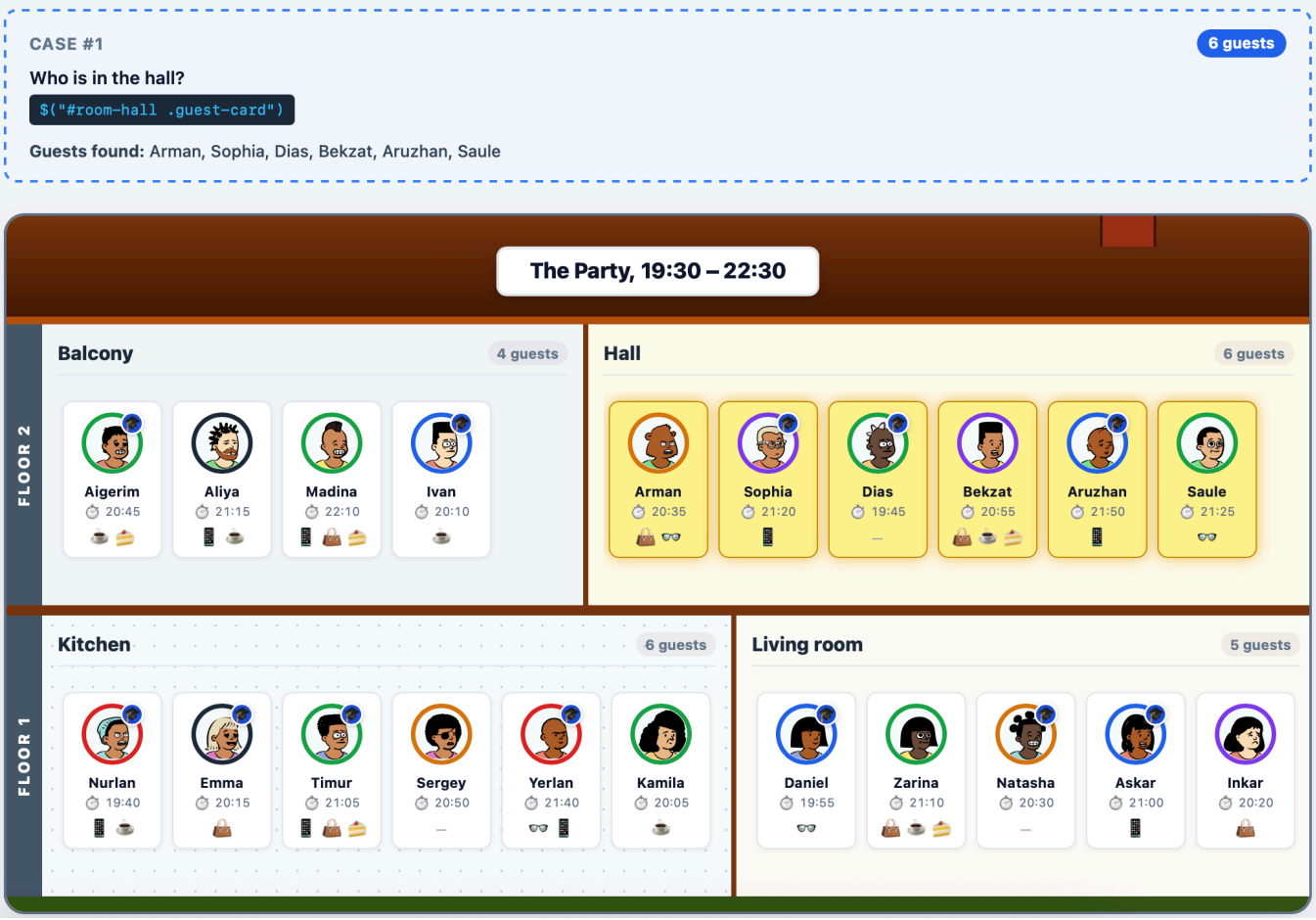


Figure: *Running party page immediately after Case 1 execution. The 6 Hall guests are highlighted with yellow glowing borders and have jumped once with animate(). The result banner displays count 6 and their respective names.*

Case 2 — Who is dressed in yellow or purple?

Employs a multiple selector combinator (comma-separated) targeting guest cards possessing either the .yellow or .purple clothing class. Retrieves 3 yellow-clothed and 3 purple-clothed guests.

Code Snippet: Actual jQuery Selector Chain:

```
$(".guest-card.yellow, .guest-card.purple")
```

Verified Result: Count: 6 guests | Identified: Arman, Sophia, Bekzat, Sergey, Natasha, Inkar

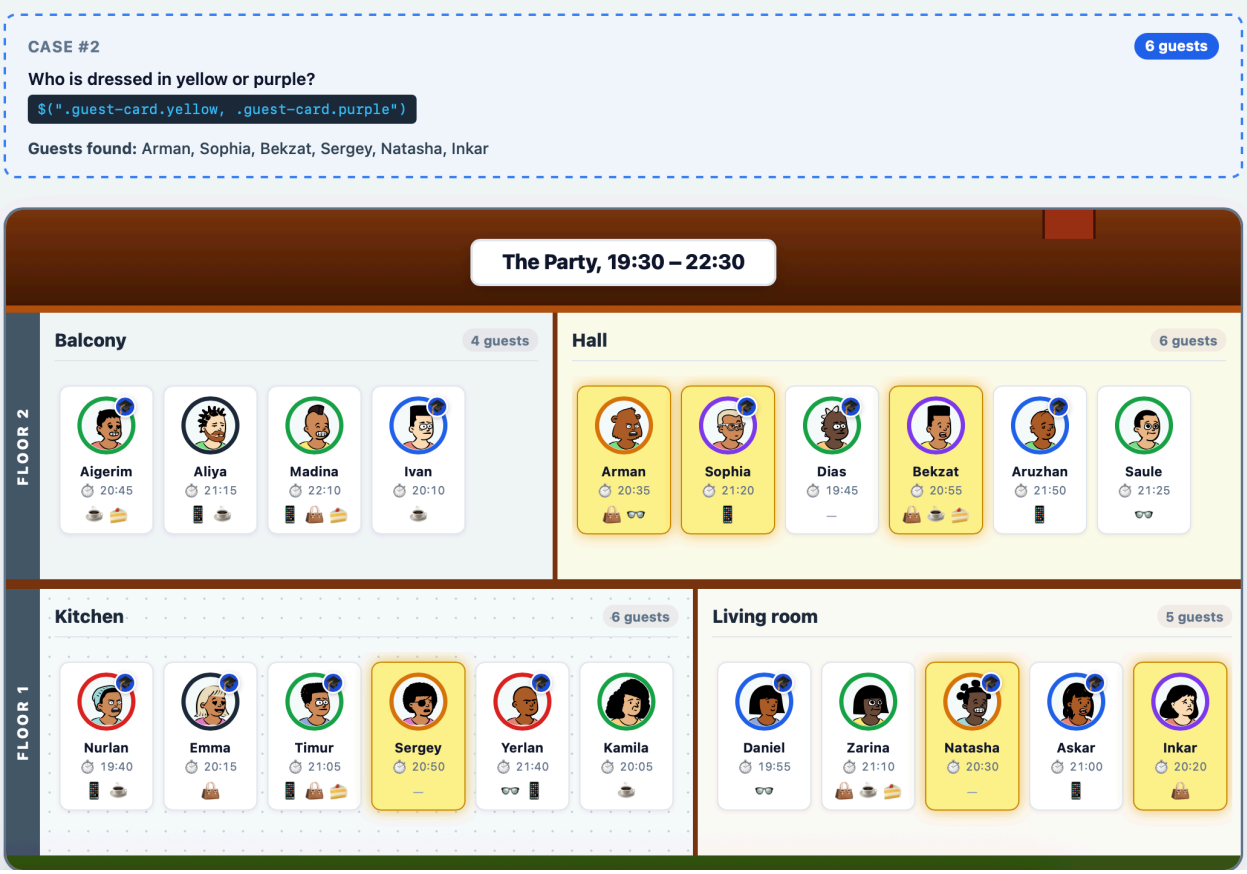


Figure: *Guest cards wearing yellow or purple rings highlighted across all rooms. Result area shows count 6 and guest names.*

Case 3 — Which AITU students are upstairs?

Combines an attribute selector targeting the second-floor container (.floor-section[data-floor='2']) with a class selector for AITU students (.guest-card.aitu). Filters out first-floor students and non-AITU upstairs guests.

Code Snippet: Actual jQuery Selector Chain:

```
$(".floor-section[data-floor='2'] .guest-card.aitu")
```

Verified Result: Count: 5 guests | Identified: Aigerim, Ivan, Sophia, Dias, Aruzhan

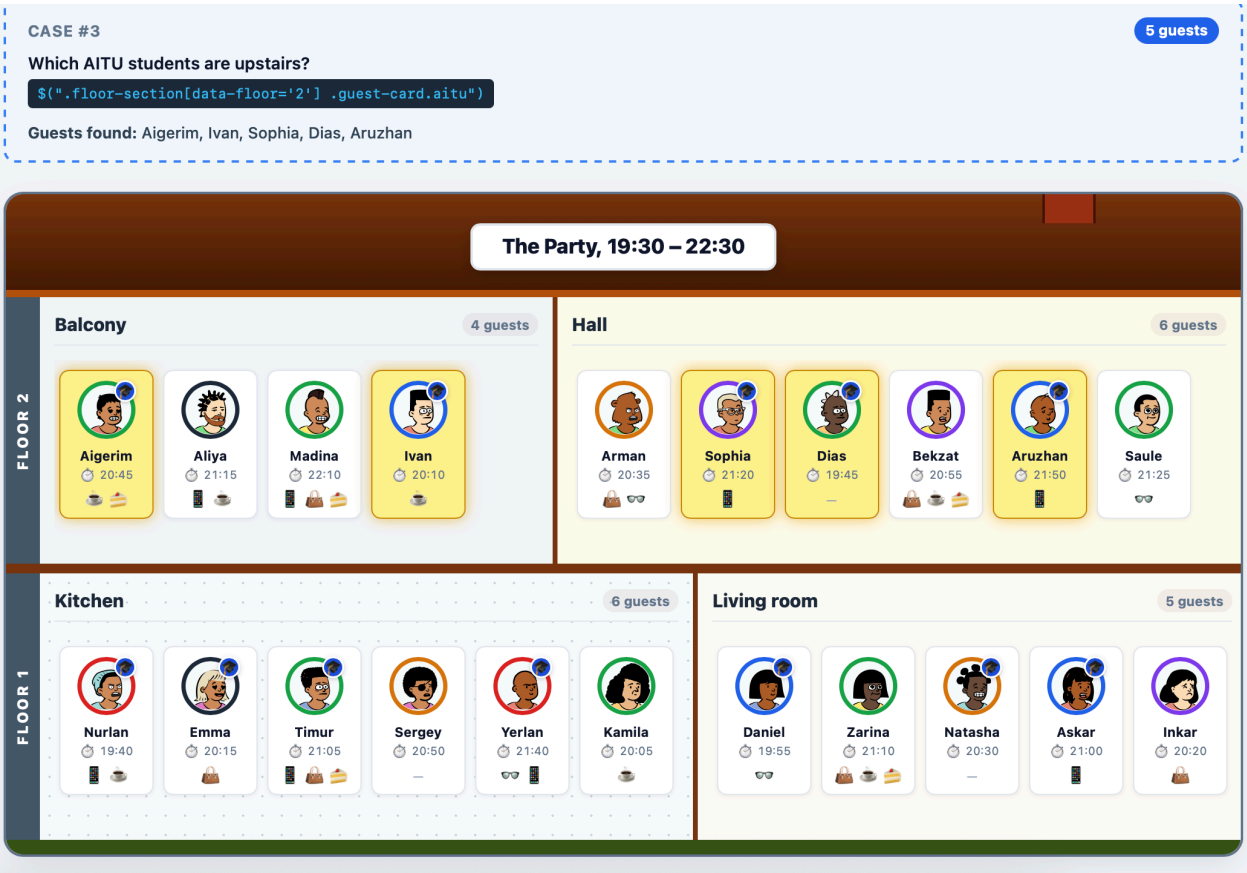


Figure: *Upstairs AITU students highlighted on Floor 2. Excludes downstairs students (Nurlan, Timur, Yerlan, Daniel, Natasha, Askar).*

Case 4 — Who has glasses or a bag?

Leverages the jQuery :has() pseudo-selector within a multiple selector to identify cards containing either glasses (.item-glasses) or a bag (.item-bag). Guests possessing both items (e.g., Arman) appear exactly once.

Code Snippet: Actual jQuery Selector Chain:

```
$(".guest-card:has(.item-glasses), .guest-card:has(.item-bag)")
```

Verified Result: Count: 10 guests | Identified: Madina, Arman, Bekzat, Saule, Emma, Timur, Yerlan, Daniel, Zarina, Inkar

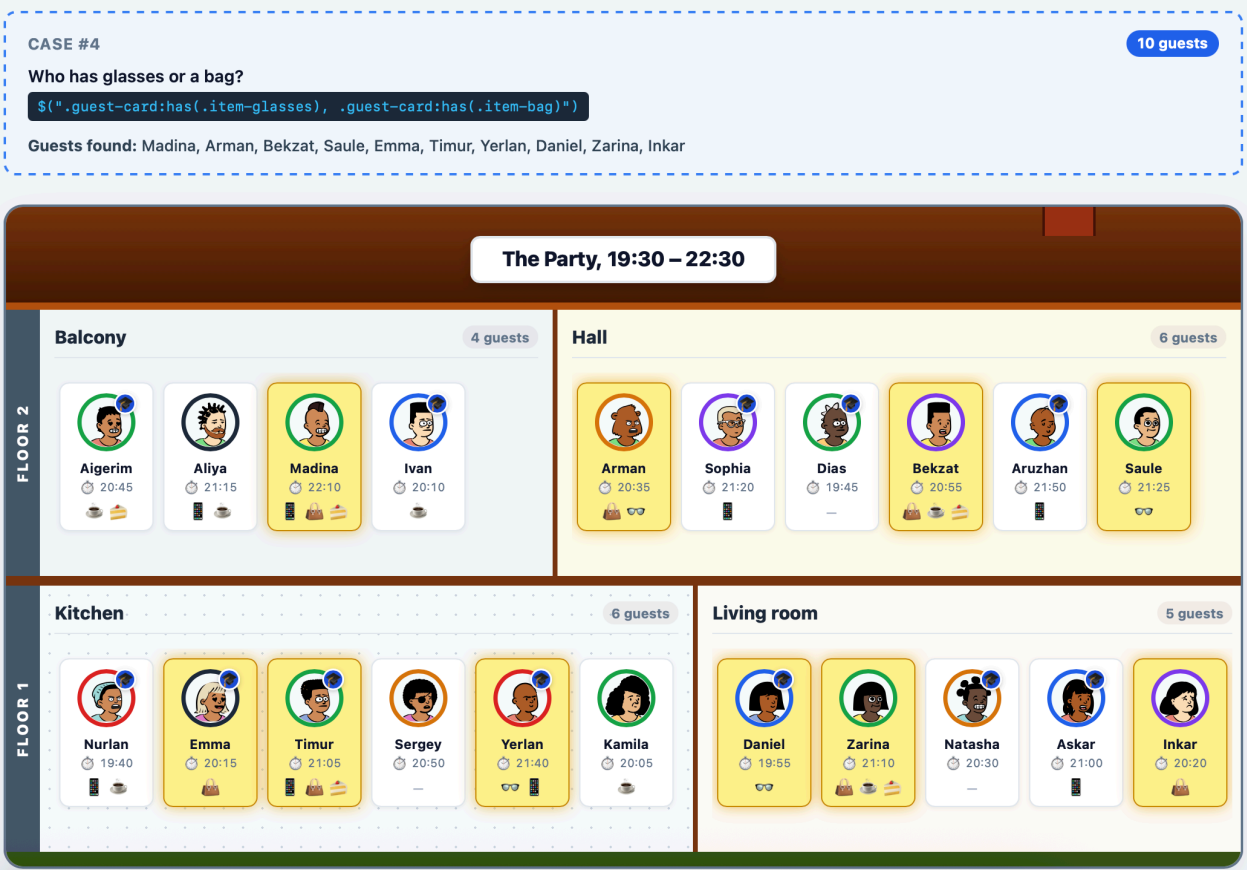


Figure: *Guests possessing glasses or bags highlighted. Verifies that cards with both items are not duplicated.*

Case 5 — Who came with nothing at all?

Combines the negation pseudo-class :not() with the hierarchical filter :has(.item-icon). Accurately distinguishes guests with zero associated items from those carrying accessories, ignoring avatar and badges.

Code Snippet: Actual jQuery Selector Chain:

```
$(".guest-card:not(:has(.item-icon))")
```

Verified Result: Count: 3 guests | Identified: Dias, Sergey, Natasha

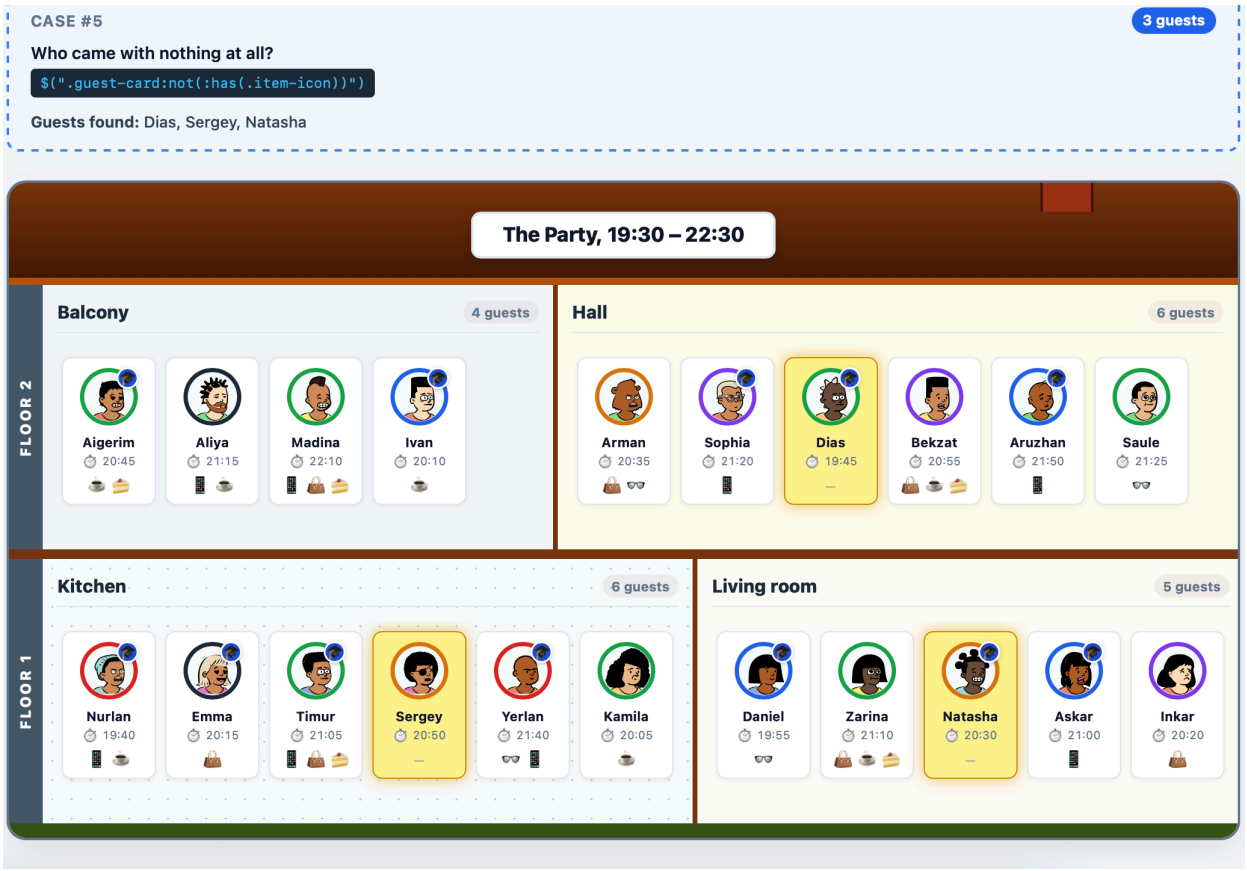


Figure: *The three guests who arrived with no accessories (Dias, Sergey, Natasha) highlighted with count 3.*

Case 6 — Who has a phone but no bag?

Chains :has(.item-phone) with :not(:has(.item-bag)) to enforce a strict logical conjunction: presence of a phone and total absence of a bag. Excludes guests carrying both (Madina, Timur).

Code Snippet: Actual jQuery Selector Chain:

```
$(".guest-card:has(.item-phone):not(:has(.item-bag))")
```

Verified Result: Count: 6 guests | Identified: Aliya, Sophia, Aruzhan, Nurlan, Yerlan, Askar

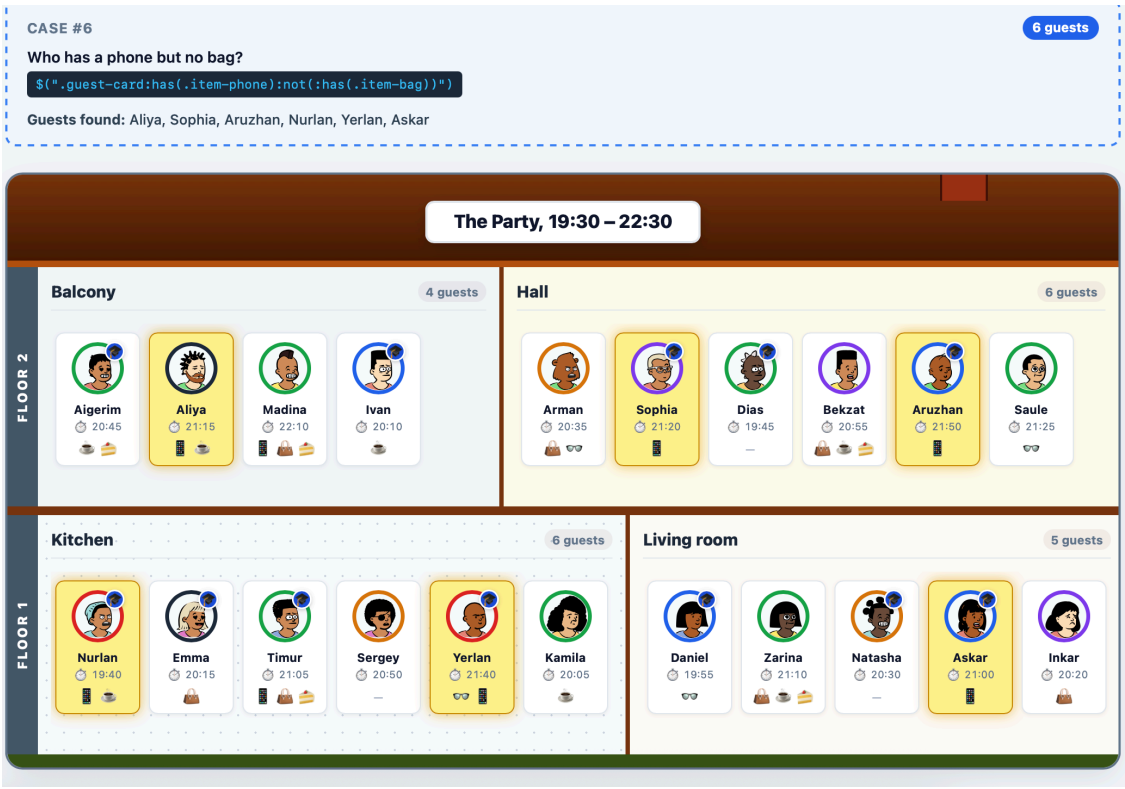


Figure: *Guests with a phone and no bag highlighted across rooms. Excludes Madina and Timur.*

Case 7 — Who arrived between 20:30 and 21:00, both times included?

Applies jQuery .filter() passing a predicate function comparing the data-arrival attribute. Inclusive inequality (>= '20:30' && <= '21:00') guarantees boundary captures (Natasha at 20:30, Askar at 21:00).

Code Snippet: Actual jQuery Selector Chain:

```
$(".guest-card").filter(function() { const t = $(this).attr("data-arrival"); return t >= "20:30" && t <= "21:00"; })
```

Verified Result: Count: 6 guests | Identified: Aigerim (20:45), Arman (20:35), Bekzat (20:55), Sergey (20:50), Natasha (20:30), Askar (21:00)

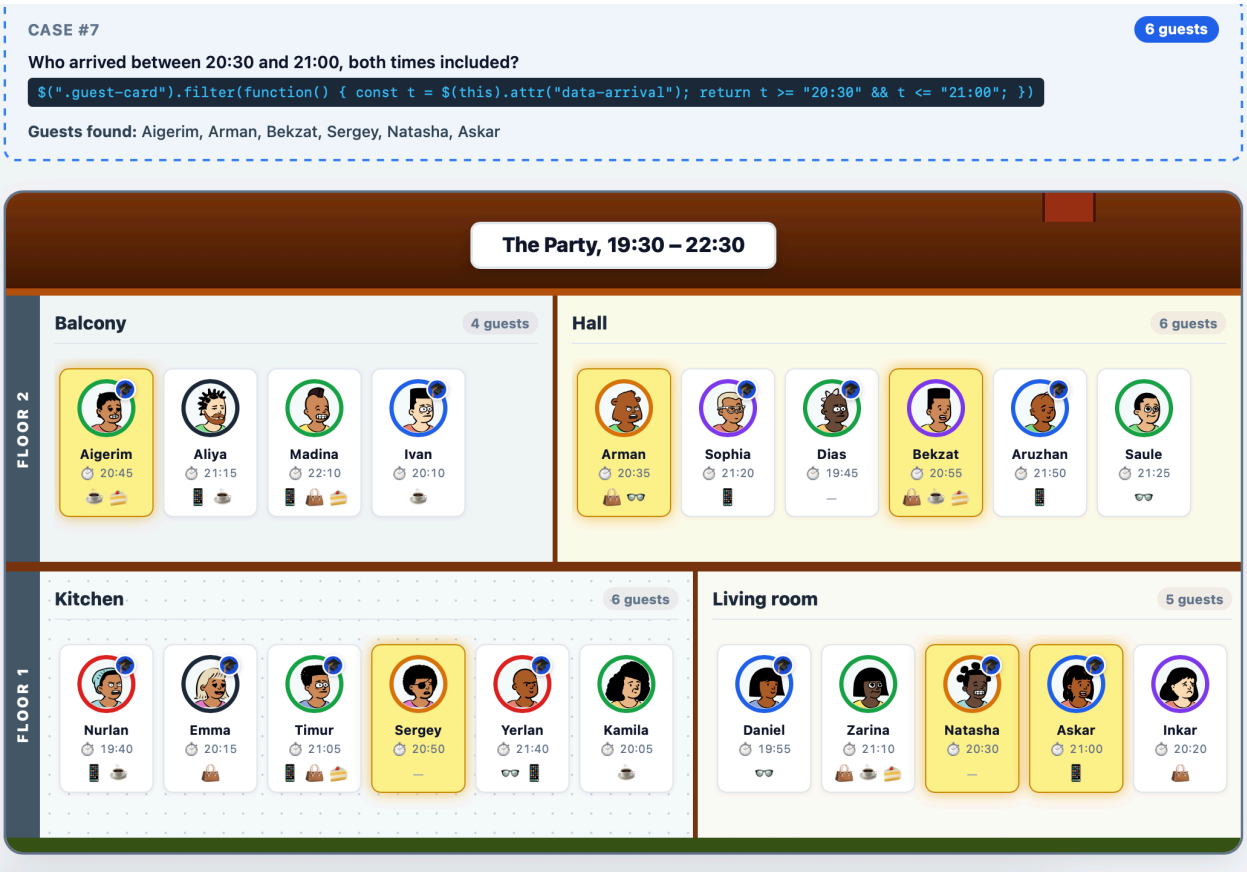


Figure: *Guests arriving in the inclusive 20:30–21:00 window highlighted. Natasha (20:30) and Askar (21:00) verified.*

Case 8 — Who arrived after 21:00 and is not an AITU student?

Pre-filters non-AITU guests using :not(.aitu), then evaluates strict inequality (> '21:00') over data-arrival. Excludes guests arriving at exactly 21:00 (Askar) and late-arriving AITU students.

Code Snippet: Actual jQuery Selector Chain:

```
$(".guest-card:not(.aitu)").filter(function() { return $(this).attr("data-arrival")
> "21:00"; })
```

Verified Result: Count: 4 guests | Identified: Aliya (21:15), Madina (22:10), Saule (21:25), Zarina (21:10)

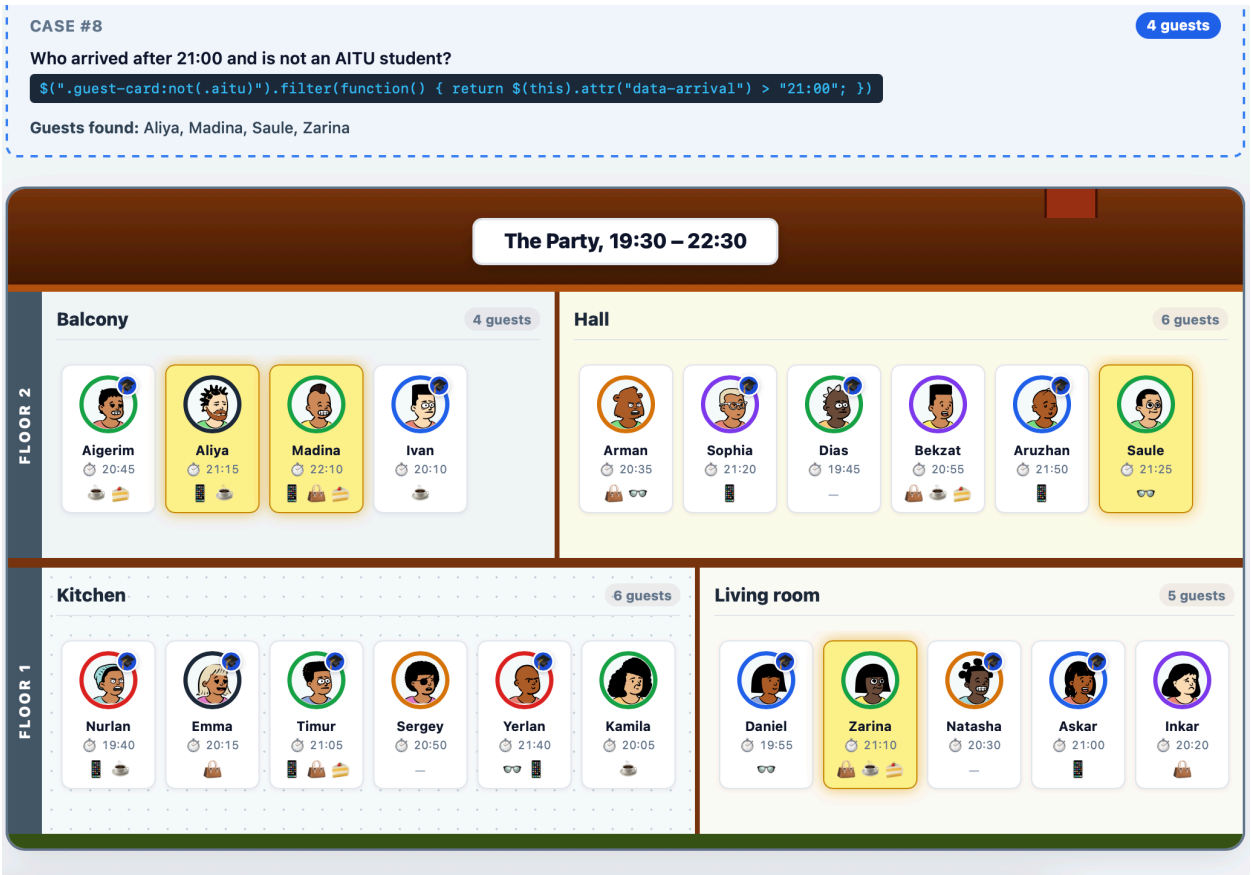


Figure: *Non-AITU guests who arrived strictly after 21:00 highlighted. Count 4 visible in result area.*

Case 9 — Who stands right next to Askar?

Locates Askar via an attribute selector, then combines his immediate predecessor (.prev()) and immediate successor (.next()) using .add(). Excludes Askar himself and non-immediate room occupants.

Code Snippet: Actual jQuery Selector Chain:

```
$(".guest-card[data-name='Askar']").prev(".guest-card").add($(".guest-card[data-name='Askar']").next(".guest-card"))
```

Verified Result: Count: 2 guests | Identified: Natasha (left), Inkar (right)

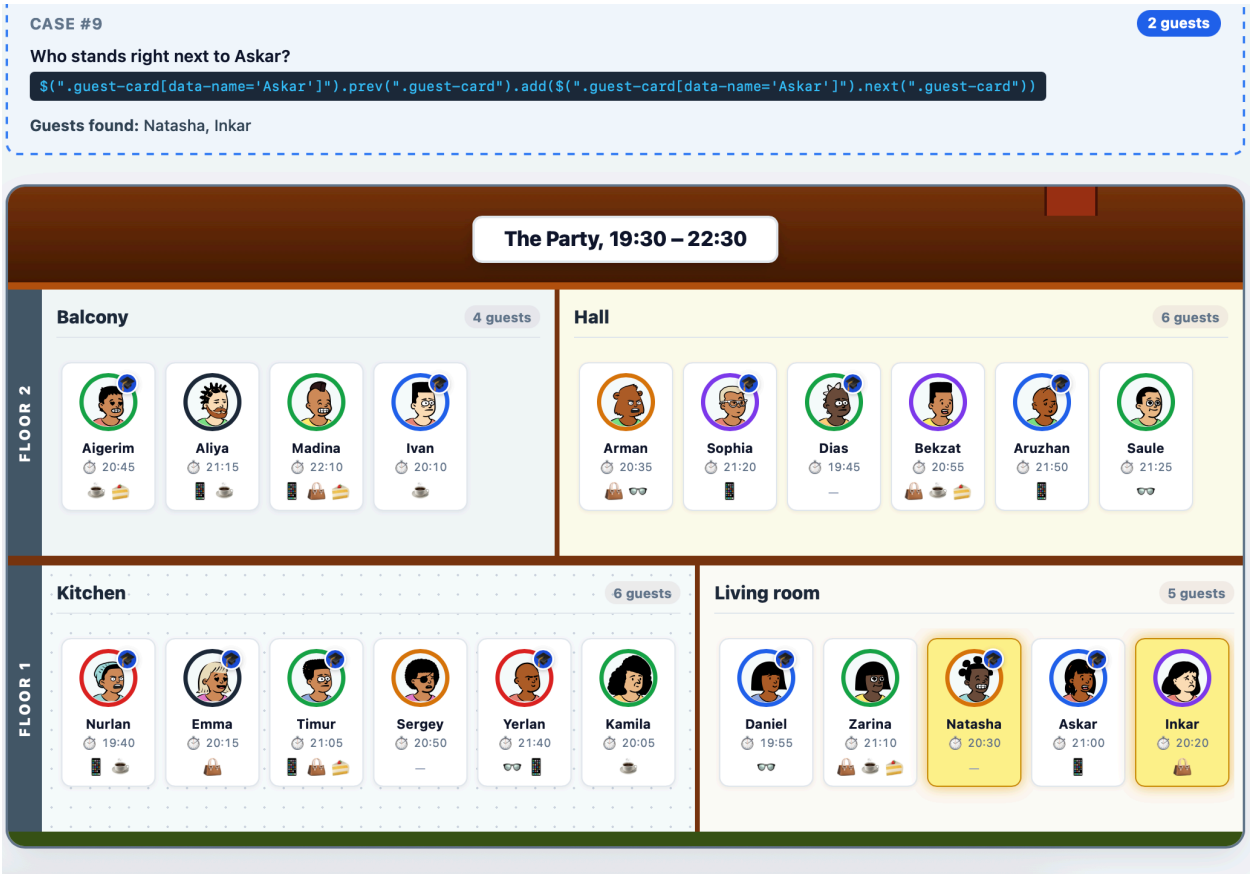


Figure: *Askar's immediate physical neighbours in the Living Room highlighted without selecting Askar.*

Case 10 — Who stands last in each room?

Utilizes the CSS structural pseudo-class :last-child scoped to room containers. Finds the rightmost card rendered in each of the four rooms.

Code Snippet: Actual jQuery Selector Chain:

```
$(".room .guest-card:last-child")
```

Verified Result: Count: 4 guests | Identified: Ivan (Balcony), Saule (Hall), Kamila (Kitchen), Inkar (Living room)

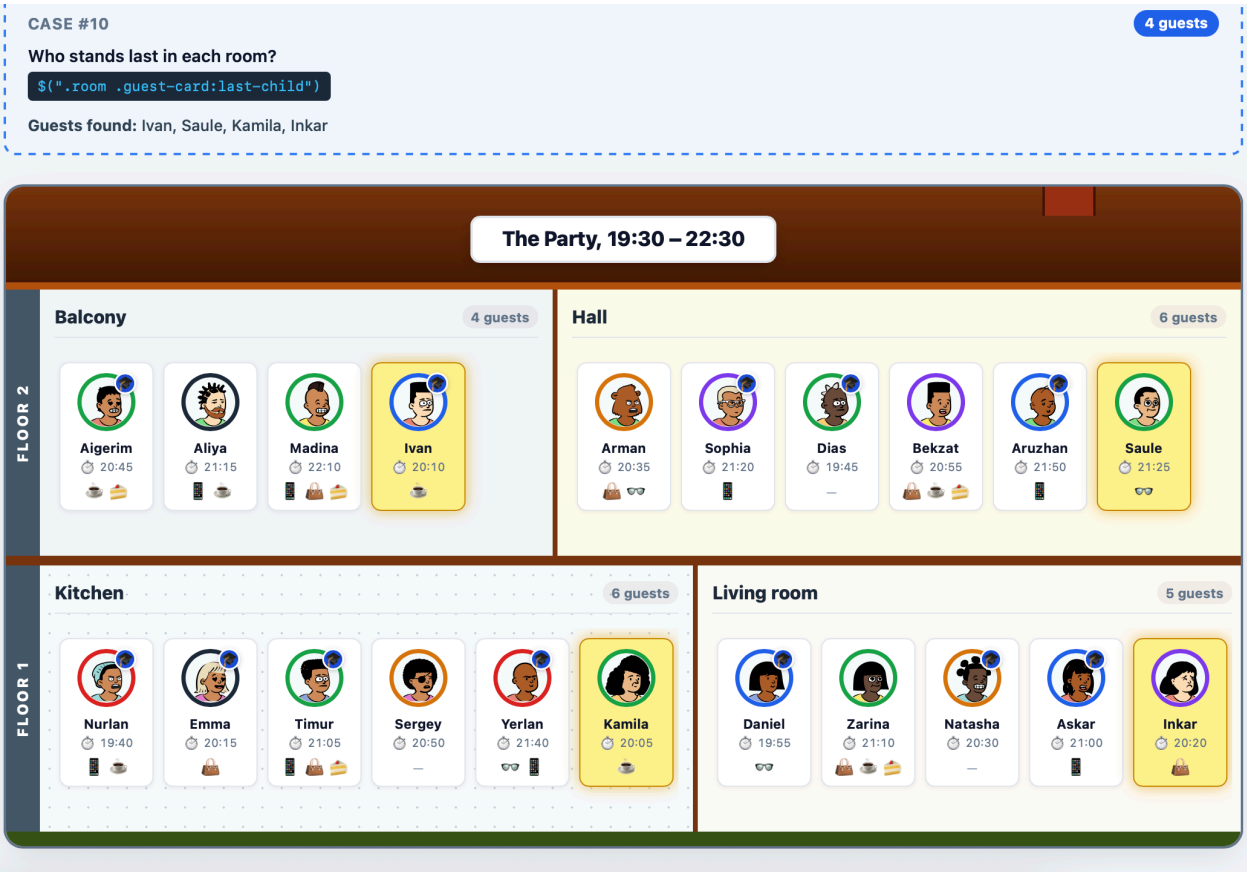


Figure: *The final card in each of the four rooms highlighted. Count 4 displayed.*

Case 11 — Who stands third in each room?

Applies :nth-child(3) scoped to each room's guest container, selecting the card at index 3 (1-based index) in left-to-right order.

Code Snippet: Actual jQuery Selector Chain:

```
$(".room .guest-card:nth-child(3)")
```

Verified Result: Count: 4 guests | Identified: Madina (Balcony), Dias (Hall), Timur (Kitchen), Natasha (Living room)

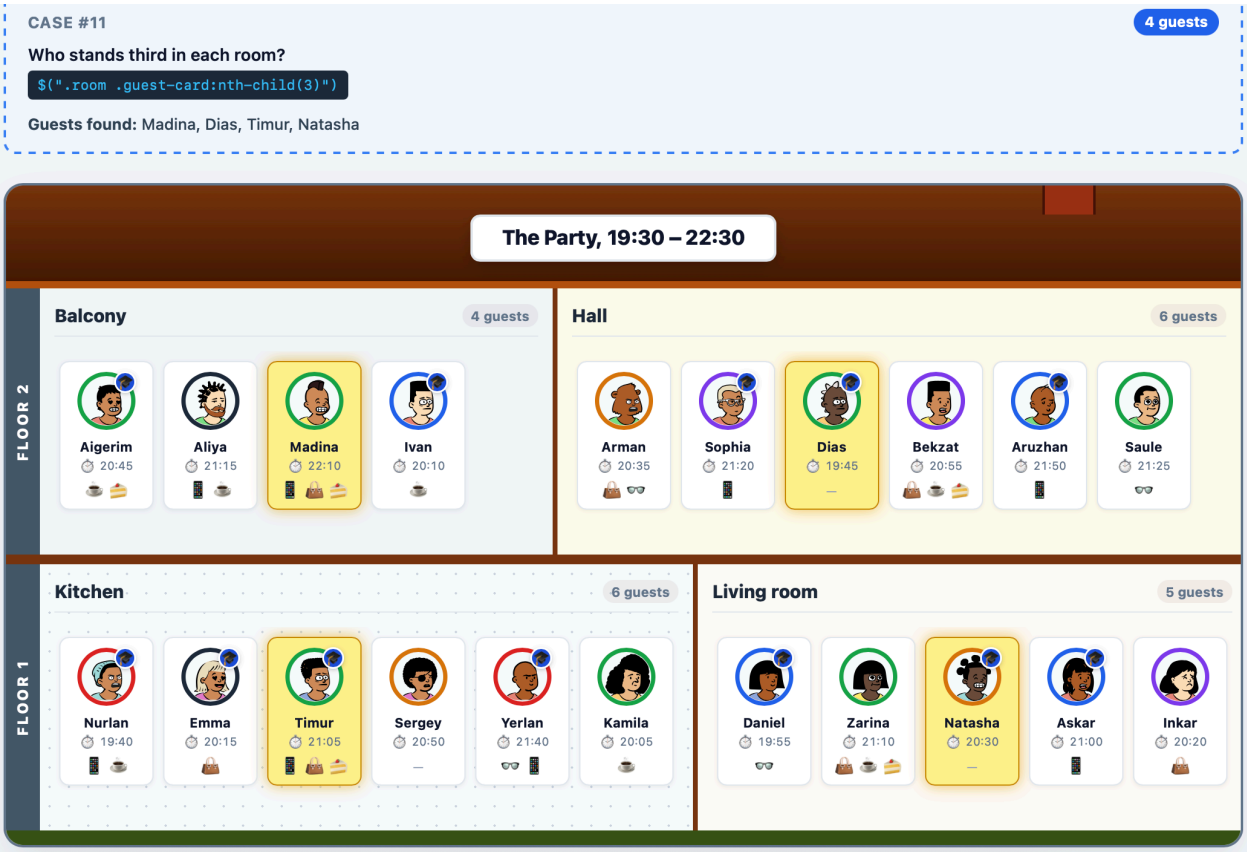


Figure: *The third card in each room highlighted across Balcony, Hall, Kitchen, and Living Room.*

Case 12 — Who stands after Timur in his room?

Demonstrates the .nextAll() traversal method starting from Timur's card, capturing all subsequent siblings in the Kitchen. Returns Sergey, Yerlan, and Kamila.

Code Snippet: Actual jQuery Selector Chain:

```
$(".guest-card[data-name='Timur']").nextAll(".guest-card")
```

Verified Result: Count: 3 guests | Identified: Sergey, Yerlan, Kamila

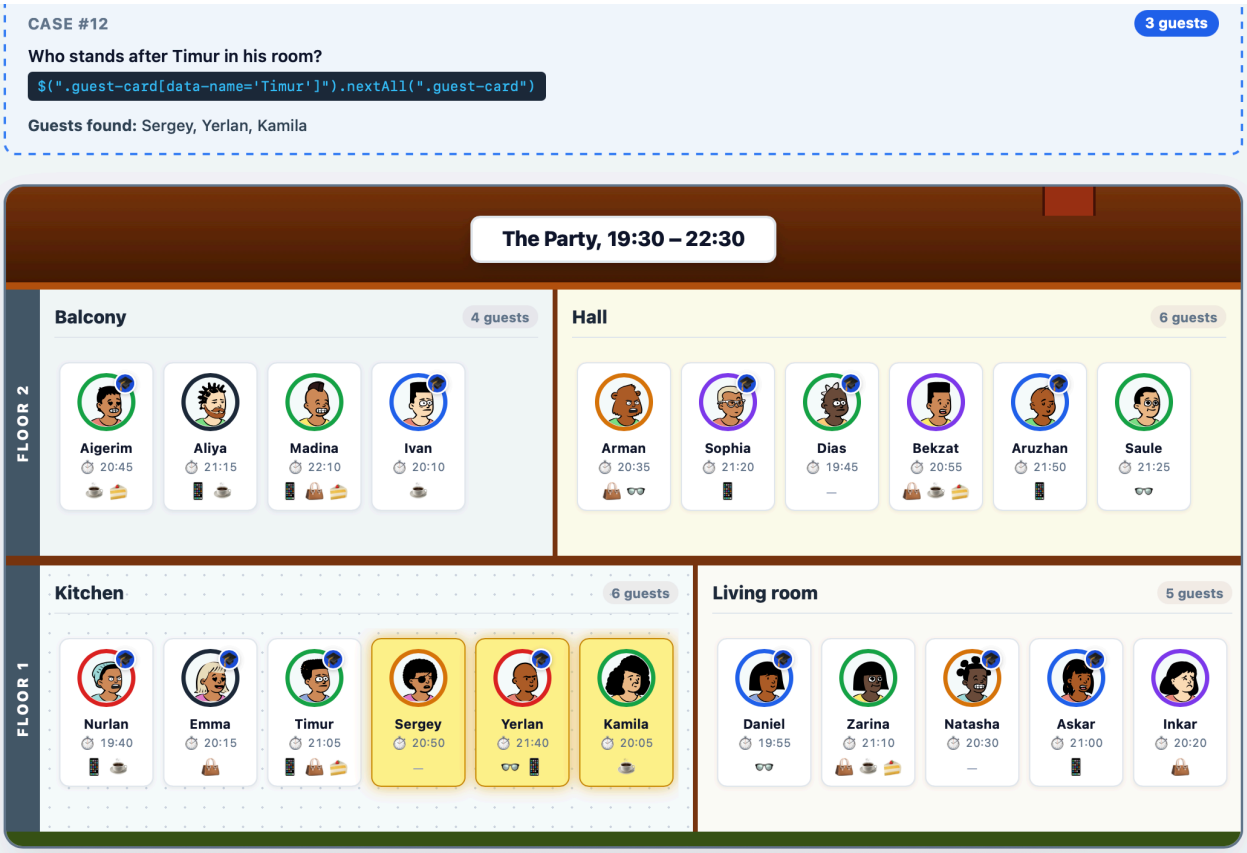


Figure: *All guests positioned after Timur in the Kitchen highlighted via nextAll(). Count 3 visible.*

Case 13 — Who stands right before someone in black?

Selects all guests wearing black (.black), then invokes .prev('.guest-card') to retrieve their immediate left predecessors within the same room container.

Code Snippet: Actual jQuery Selector Chain:

```
$(".guest-card.black").prev(".guest-card")
```

Verified Result: Count: 2 guests | Identified: Aigerim (before Aliya in Balcony), Nurlan (before Emma in Kitchen)

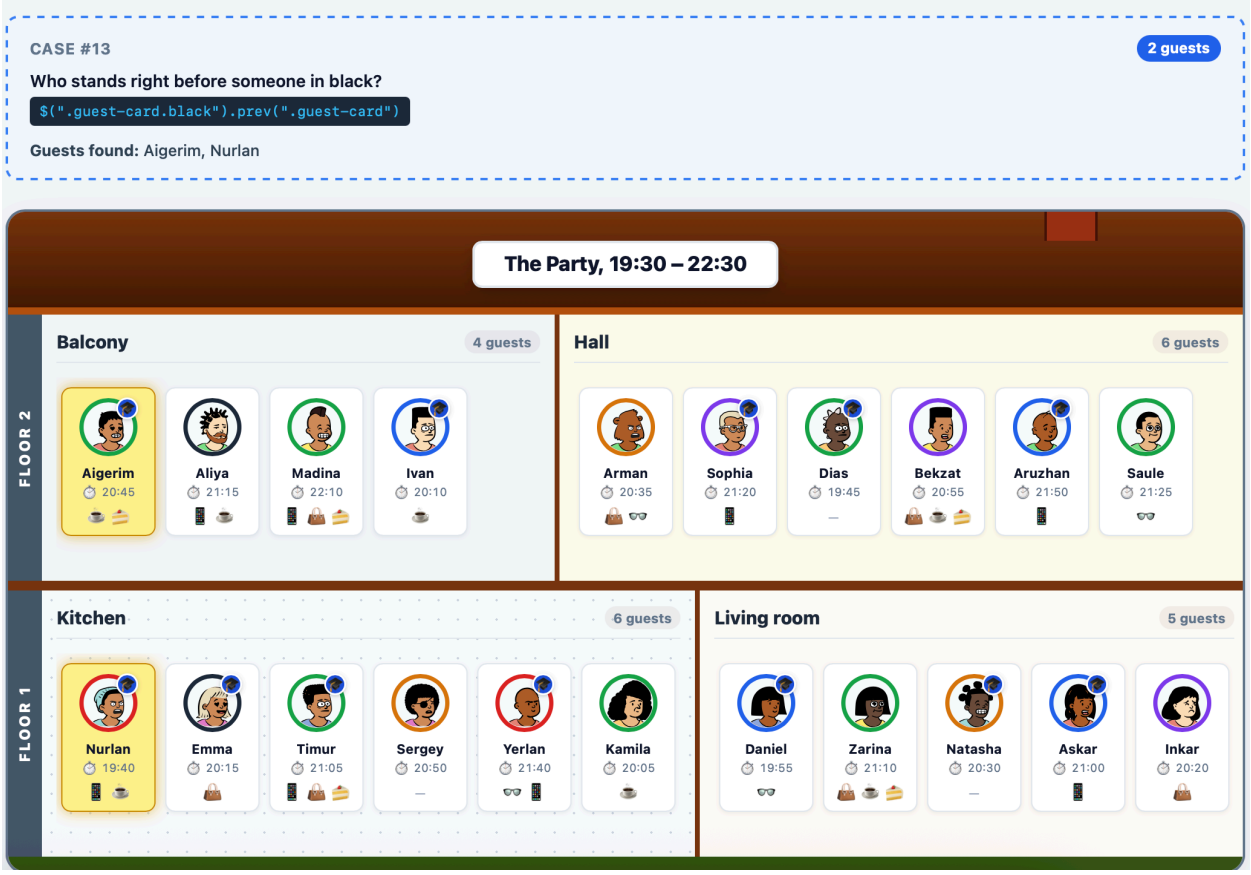


Figure: *Immediate predecessors of black-clothed guests highlighted. Aigerim and Nurlan identified.*

Case 14 — In which room is Aruzhan?

Traverses upward from Aruzhan's card to its parent room container using .closest('.room'). Highlights the Hall room element and reports room identity rather than guest card.

Code Snippet: Actual jQuery Selector Chain:

```
$(".guest-card[data-name='Aruzhan']").closest(".room")
```

Verified Result: Count: 1 room | Identified: Hall (Floor 2)

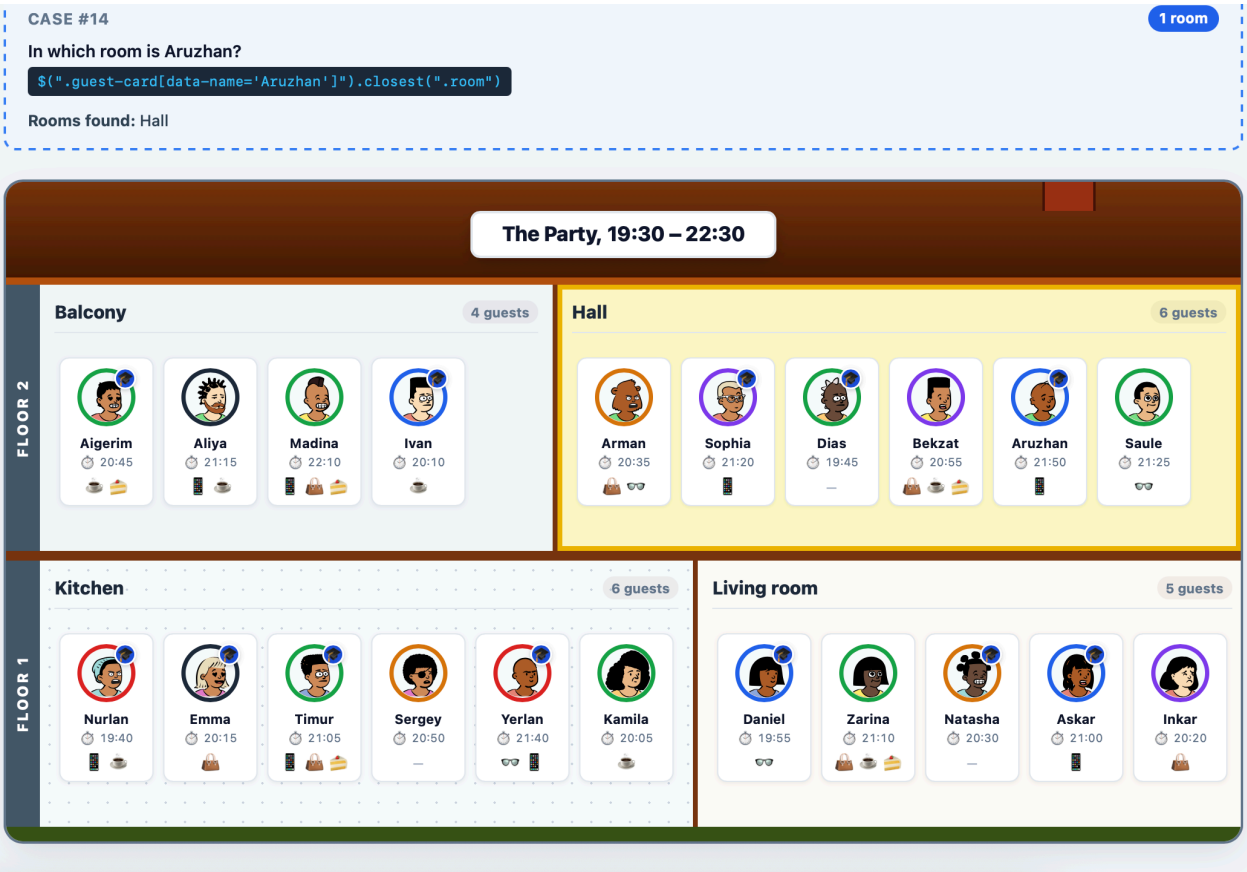


Figure: *The Hall room container highlighted with yellow border; guests inside Hall jump once. Result banner displays '1 room: Hall'.*

Case 15 — Which rooms have fewer than 6 guests?

Filters the four room elements using .filter() by evaluating the length of $(this).find('.guest-card'). Balcony has 4 guests and Living Room has 5 guests, satisfying < 6.

Code Snippet: Actual jQuery Selector Chain:

```
$(".room").filter(function() { return $(this).find(".guest-card").length < 6; })
```

Verified Result: Count: 2 rooms | Identified: Balcony (4 guests), Living room (5 guests)

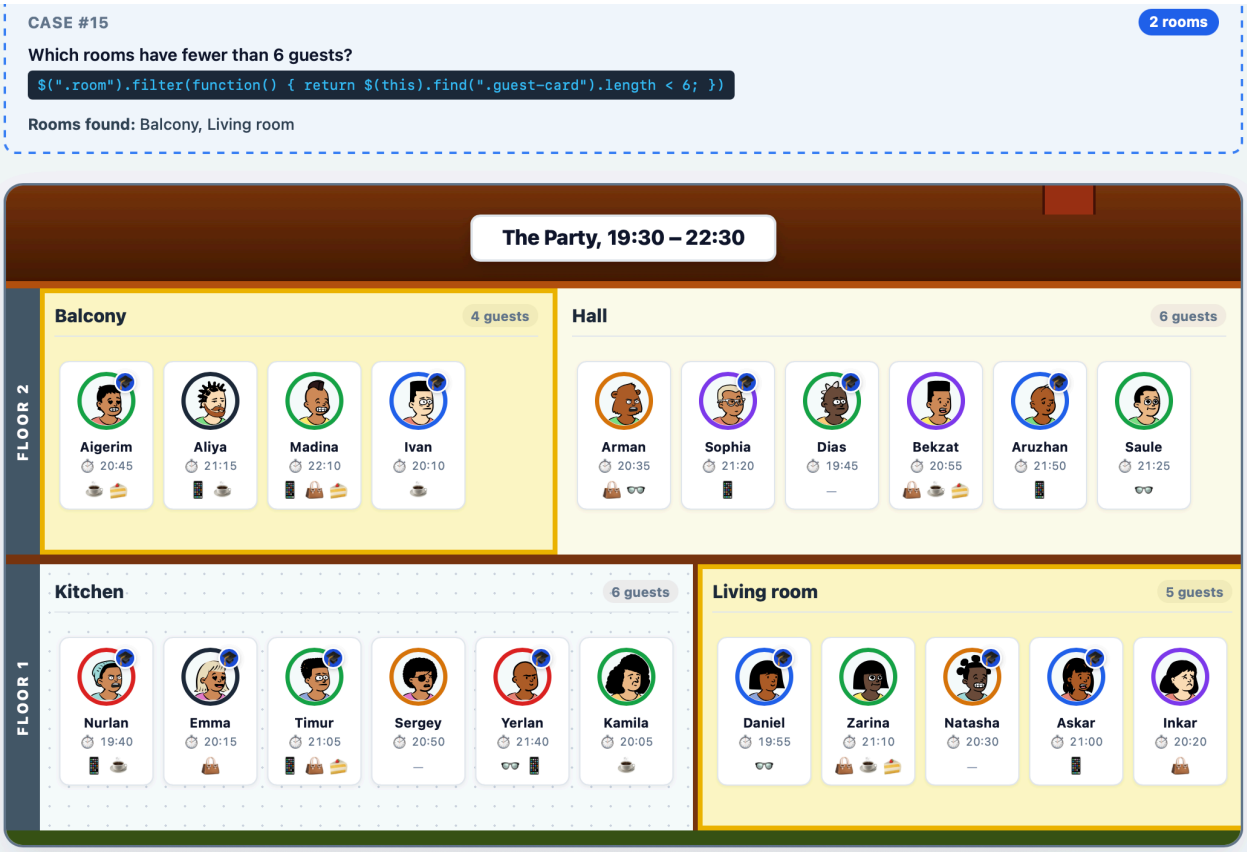


Figure: *Balcony and Living room highlighted. Count 2 rooms displayed.*

Case 16 — In which rooms does somebody have a cup, but nobody wears glasses?

Evaluates rooms possessing at least one cup (:has(.item-cup)) while strictly having zero pairs of glasses (:not(:has(.item-glasses))). Evaluates independently per room; only Balcony satisfies both.

Code Snippet: Actual jQuery Selector Chain:

```
$(".room:has(.item-cup):not(:has(.item-glasses))")
```

Verified Result: Count: 1 room | Identified: Balcony (3 cups, 0 glasses)

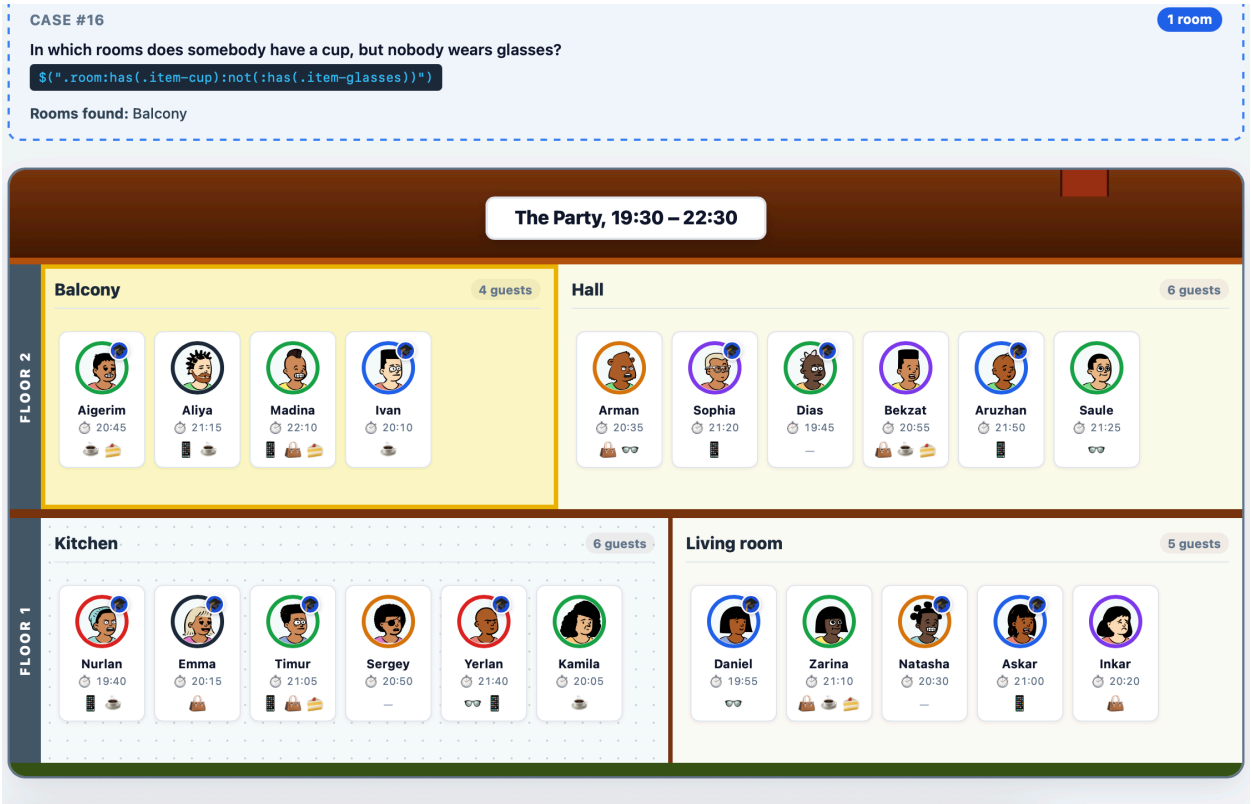


Figure: *Balcony room highlighted as the sole room meeting the compound constraint. Result area shows Balcony.*

Case 17 — Who came first to each room?

Iterates over rooms, evaluating data-arrival across child cards to find the chronological minimum timestamp in each room. Returns Ivan (20:10), Dias (19:45), Nurlan (19:40), and Daniel (19:55).

Code Snippet: Actual jQuery Selector Chain:

```
$(".room").map(function() { let e = null, min = "99:99"; $(this).find(".guest-card").each(function() { const t = $(this).attr("data-arrival"); if (t < min) { min = t; e = this; } }); return e; })
```

Verified Result: Count: 4 guests | Identified: Ivan (20:10), Dias (19:45), Nurlan (19:40), Daniel (19:55)

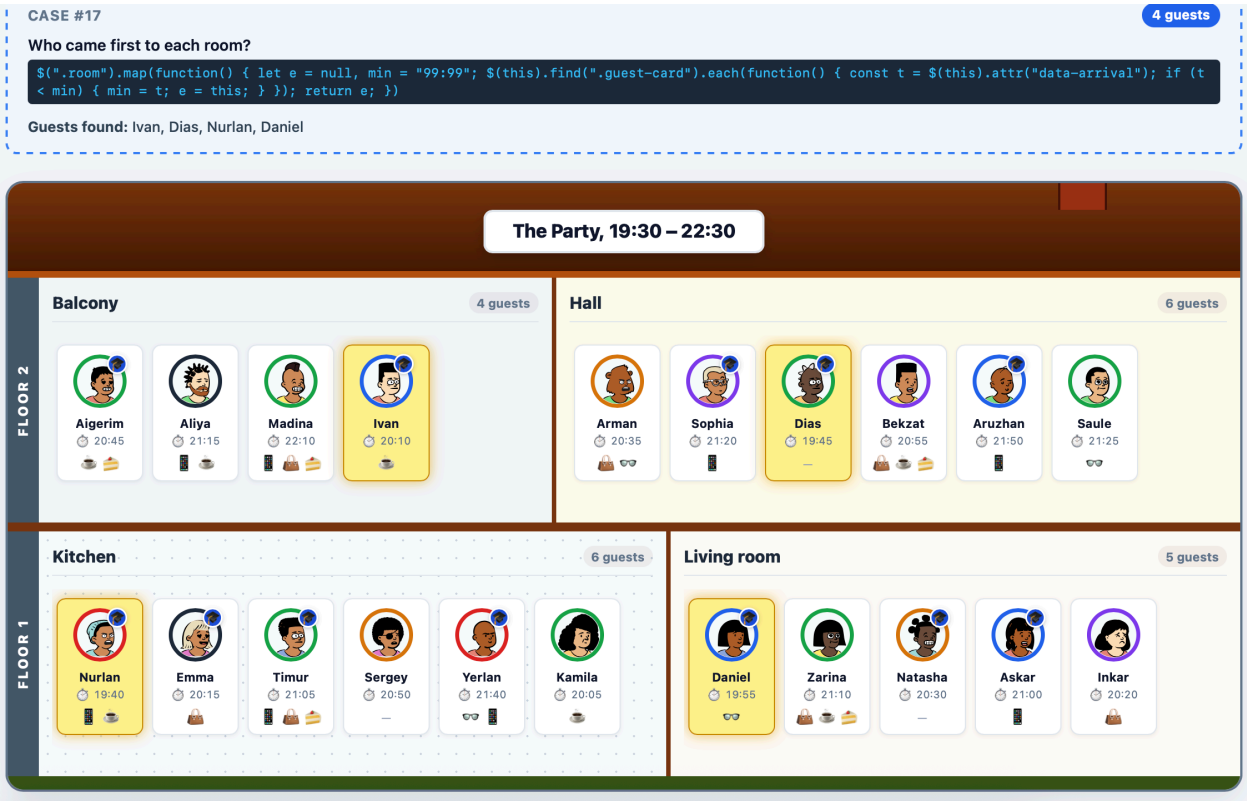


Figure: *The earliest arrival in every room highlighted. Real timestamps parsed and compared.*

Case 18 — Miras joins Askar and Inkar

Rebuilds the initial party, instantiates a valid Miras card (21:20, blue clothing, AITU badge, phone), inserts him between Askar and Inkar using insertAfter(), and smoothly reveals him with fadeIn(600).

Code Snippet: Actual jQuery Selector Chain:

```
const mirasCard = createGuestCard(mirasData).hide(); mirasCard.insertAfter($
(".guest-card[data-name='Askar']")).fadeIn(600);
```

Verified Result: Count: 1 guest added | Identified: Miras (Living room, between Askar and Inkar)



Figure: *Miras card rendered between Askar and Inkar in the Living Room following smooth fadeIn(600). Card exhibits blue ring, 21:20 arrival, phone icon, and AITU badge.*

Case 19 — Natasha leaves the party

Rebuilds initial party, locates Natasha's card, applies a goodbye highlight, animates her departure via fadeOut(600), and removes the DOM element upon completion using .remove().

Code Snippet: Actual jQuery Selector Chain:

```
const natasha = $(".guest-card[data-name='Natasha']"); natasha.fadeOut(600, function() { $(this).remove(); });
```

Verified Result: Count: 1 guest departed | Identified: Natasha (card removed from Living room)

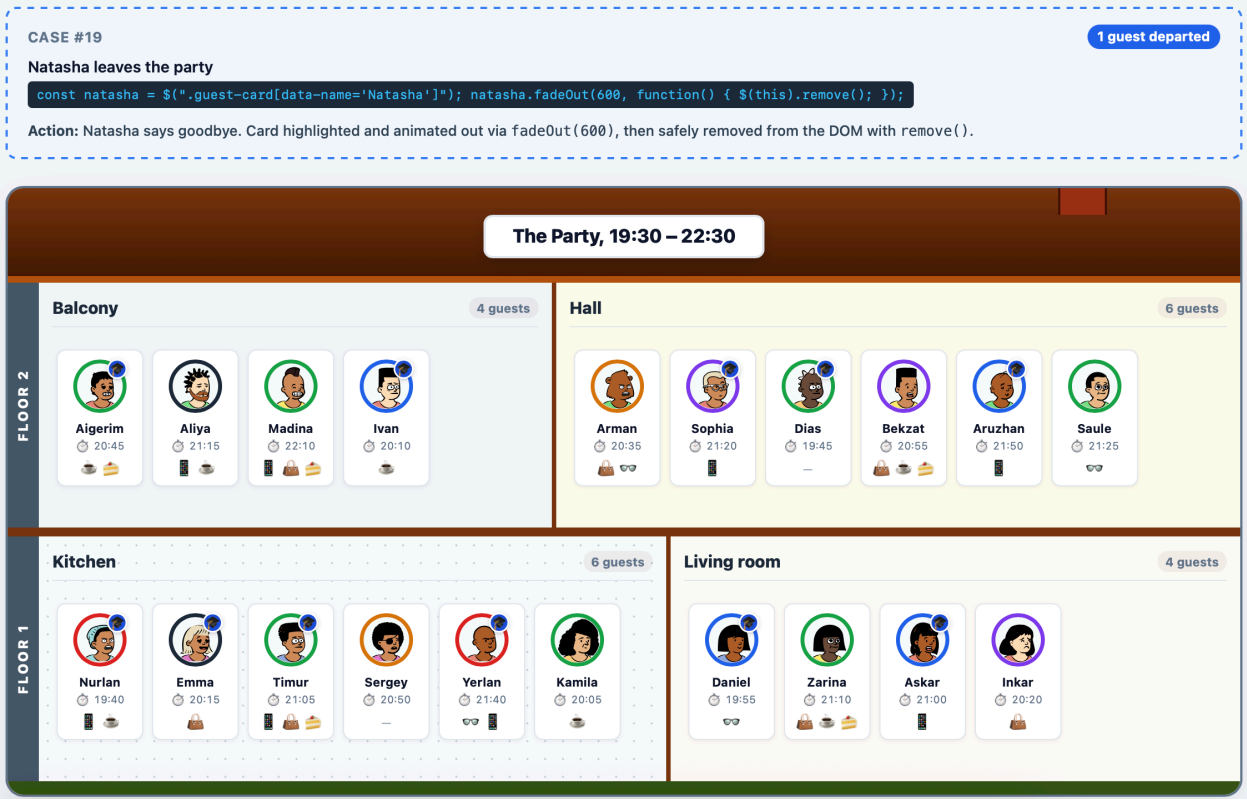


Figure: *Living Room after Natasha's departure. Card completely removed from DOM; no ghost element remains.*

Case 20 — Who took the cake?

Evaluates the complete 4-clue logical conjunction against the DOM: (1) Downstairs (.floor-section[data-floor='1']), (2) Strict arrival window 21:00 < t < 21:30, (3) Possesses cake crumbs without a cup, and (4) Stood immediately adjacent to an AITU student (.prev('.aitu') or .next('.aitu')). Uniquely unmasks Timur.

Code Snippet: Actual jQuery Selector Chain:

```
$(".floor-section[data-floor='1'] .guest-card").filter(":has(.item-
crumbs):not(:has(.item-cup))").filter(function() { const t = $(this).attr("data-
arrival"); return t > "21:00" && t < "21:30"; }).filter(function() { return $
(this).prev(".aitu").length > 0 || $(this).next(".aitu").length > 0; })
```

Verified Result: Count: 1 thief identified | Identified: Timur (Kitchen, Floor 1)

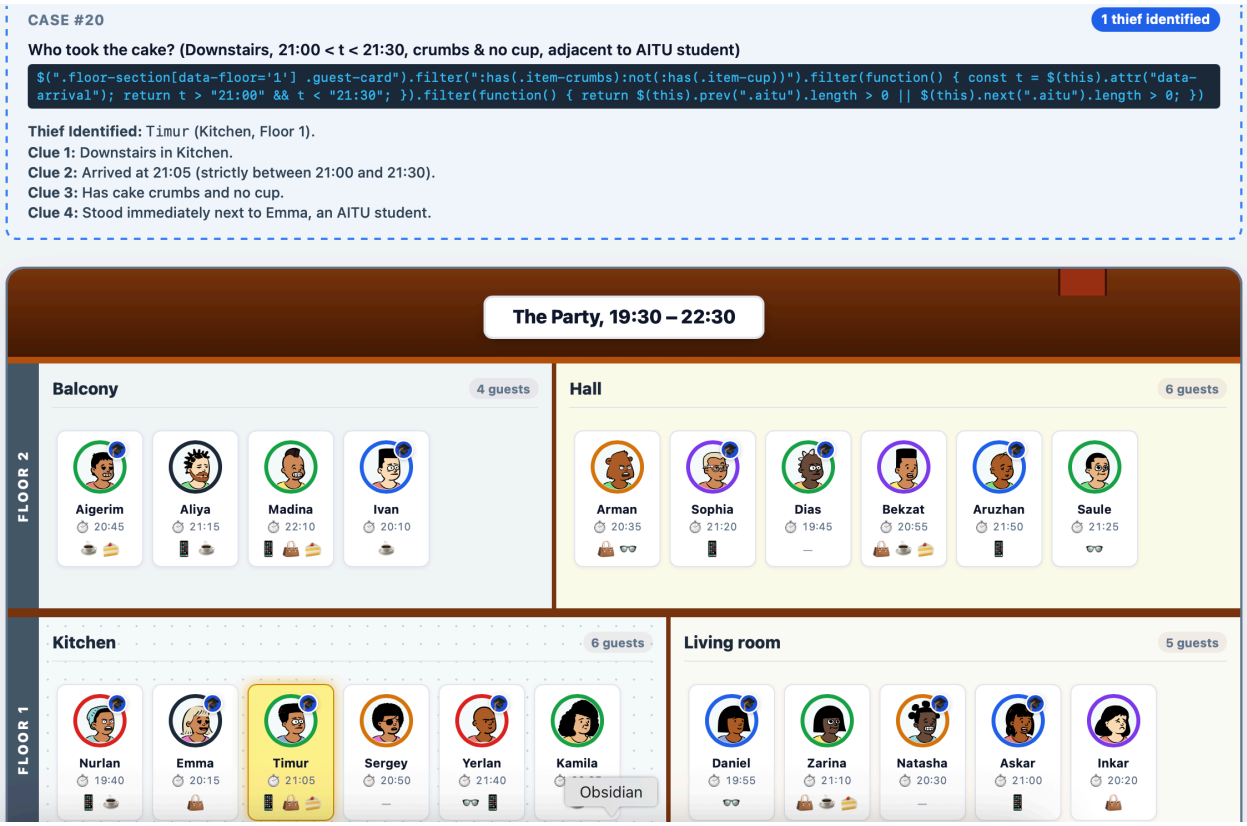


Figure: *Timur highlighted in the Kitchen as the cake thief. Result banner explains how all four clues converge strictly and uniquely on him.*

4. Task 2 — The Party in Motion (Effects & Animations)

Task 2 develops an independent copy of the party web application that brings the interface to life through interactive jQuery animations. Unlike Task 1, Task 2 preserves mutable party state: moving a guest updates both the live DOM and the underlying data array, so that subsequent queries reflect the moved guest's new location. CSS transitions and CSS animations remain strictly prohibited.

4.1. Effect 1 — Spotlight & Clear (fadeTo)

Clicking any case button highlights matching elements while dimming unrelated cards. Target cards remain fully opaque (opacity: 1), while non-target cards smoothly fade to 20% opacity over 300 milliseconds using jQuery's fadeTo(300, 0.2). When a room query executes (Cases 14–16), all guests in the matched rooms remain bright, and guests in non-matching rooms fade. Clicking 'Clear Spotlight' executes fadeTo(300, 1.0) across all guest cards and removes highlight borders.

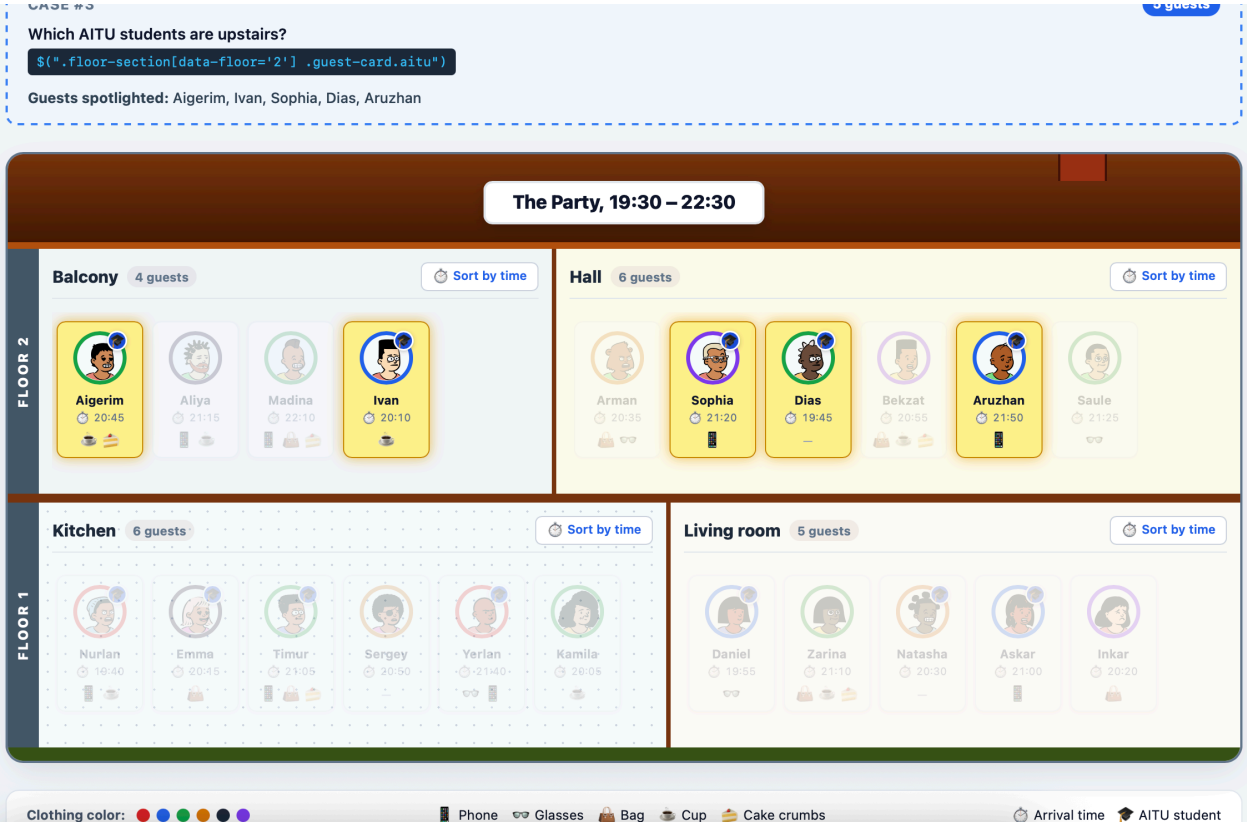


Figure: *Running Task 2 after clicking Case 3. Upstairs AITU students on Floor 2 remain at full opacity (1.0), while all other guest cards across the house fade to 20% opacity (0.2).*

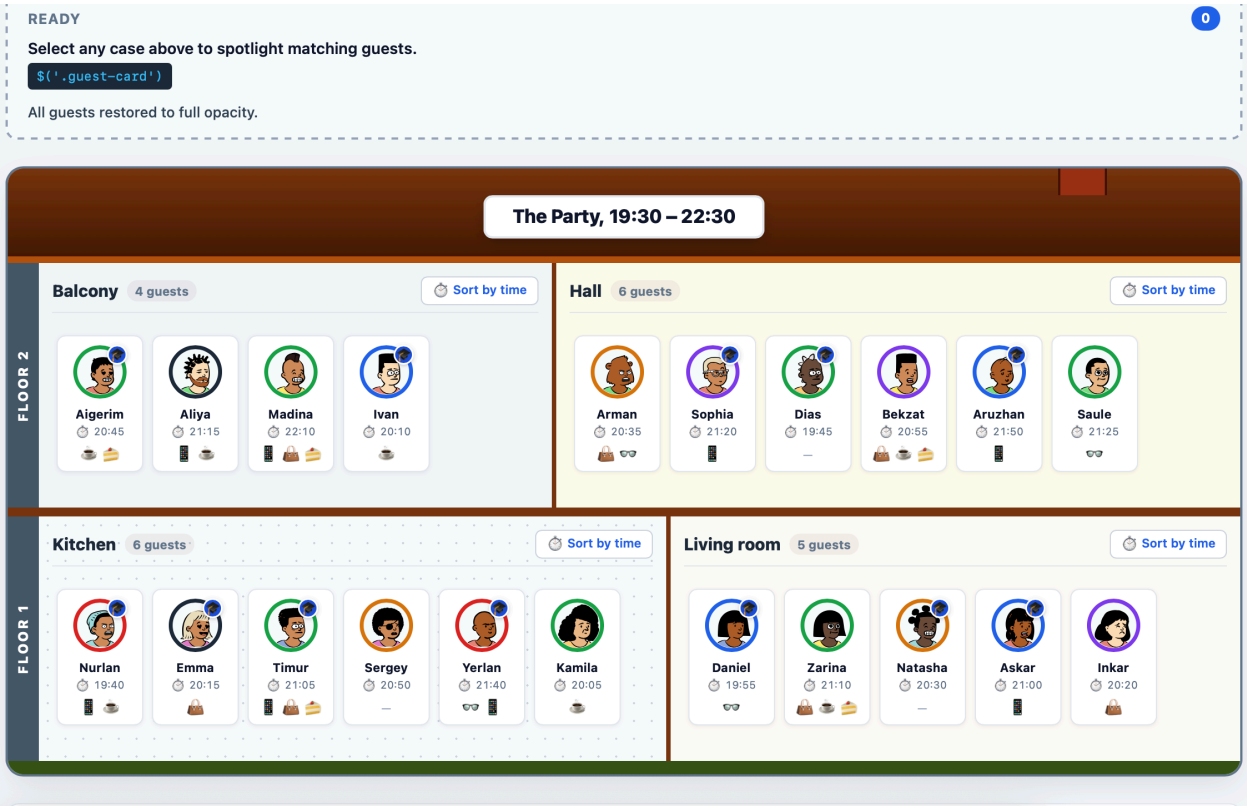


Figure: *Running Task 2 after clicking Clear Spotlight. All guest cards are restored to full opacity (1.0) and highlight styling is cleared.*

4.2. Effect 2 — Expanding Guest Detail Card

Clicking any guest card dynamically spawns an interactive modal card displaying full guest information formatted in descriptive words: arrival time ('Came at 20:30.'), student status ('AITU student.'), accessories ('Has a phone and a bag.'), clothing ('Wears yellow.'), and current room ('Currently in the Living room (Floor 1).').

- • Opening Animation: The card does not abruptly appear. Using $avatar.offset(), the modal begins at the avatar's exact coordinates and dimensions (48x48px, opacity 0.2) and expands outward to centered modal dimensions (360px wide, opacity 1.0) over 300ms via jQuery animate().
- • Closing Animation: Triggered when pressing Escape, clicking the backdrop outside the card, or clicking the close button. The card animates shrinking back into the originating avatar's coordinates over 240ms, safely removing the element upon completion.

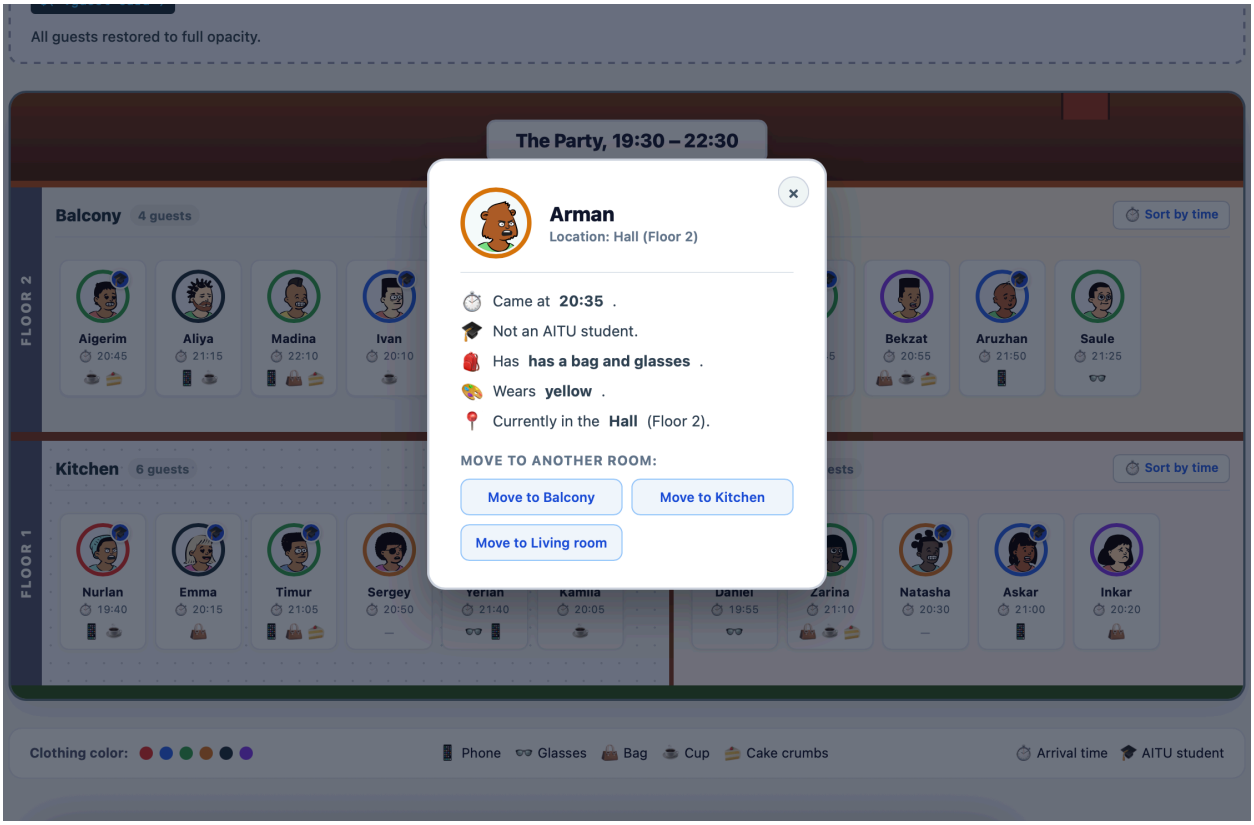


Figure: *Expanded guest detail card opened after clicking Arman. Demonstrates outward expansion from his avatar, clear descriptive sentences, and room movement buttons.*

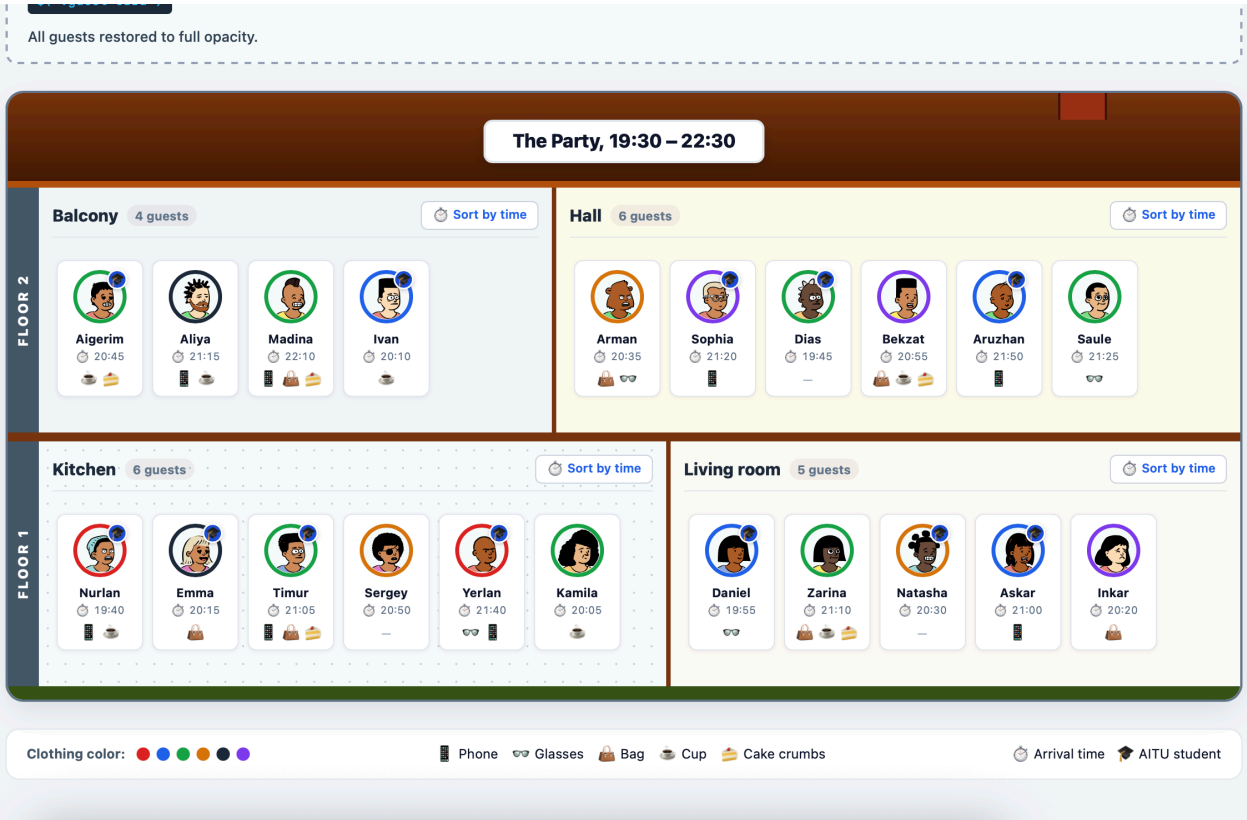


Figure: *View immediately after pressing Escape. The detail card has shrunk back into Arman's avatar and the overlay is cleanly removed from the DOM.*

4.3. Effect 3 — Cross-House Flying Room Transfers

Inside the guest detail card, action buttons allow moving the guest to any other room (e.g., 'Move to Balcony'). When clicked, the transfer executes as a continuous physical flight rather than an abrupt teleportation:

- **1.** Placeholder Reservation: An animated placeholder (.guest-card-placeholder) is appended to the target room's list, expanding its width from 0 to 78px over 280ms to smoothly open space.
- **2.** Flight Animation: A visual clone (.flying-guest-card) is created at the card's starting offset and animated across the screen via animate({ top: dest.top, left: dest.left }, 500). Simultaneously, the original card in the old room animates its width to 0, smoothly closing the vacated space.
- **3.** State Synchronization: Upon arrival, the clone and placeholder are removed, the real card is placed at the end of the destination room with updated data-room and data-floor attributes, and the guest object is transferred between arrays in the liveParty data structure.

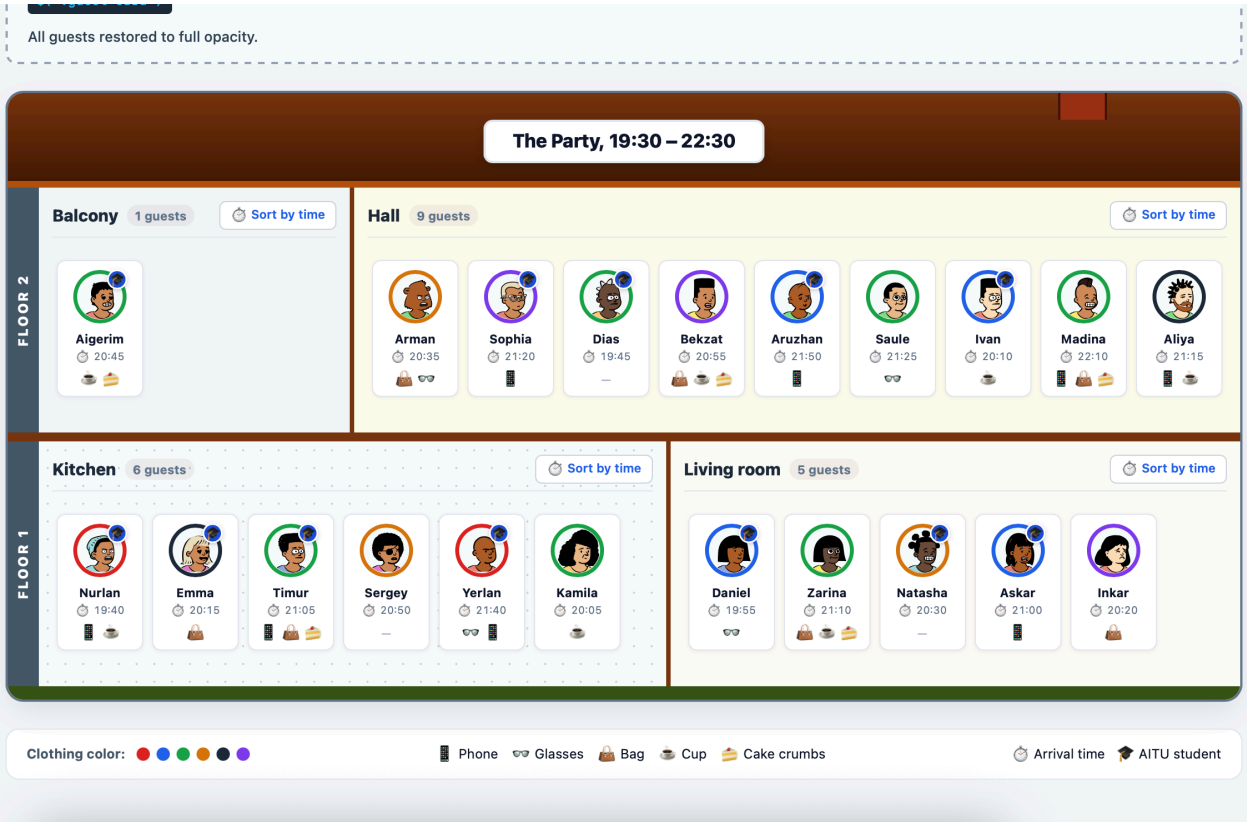


Figure: *Arman positioned at the right end of the Balcony following cross-house flight. The Hall gap is completely closed, Balcony room badge reads '5 guests', and Arman's attributes reflect Floor 2 Balcony.*

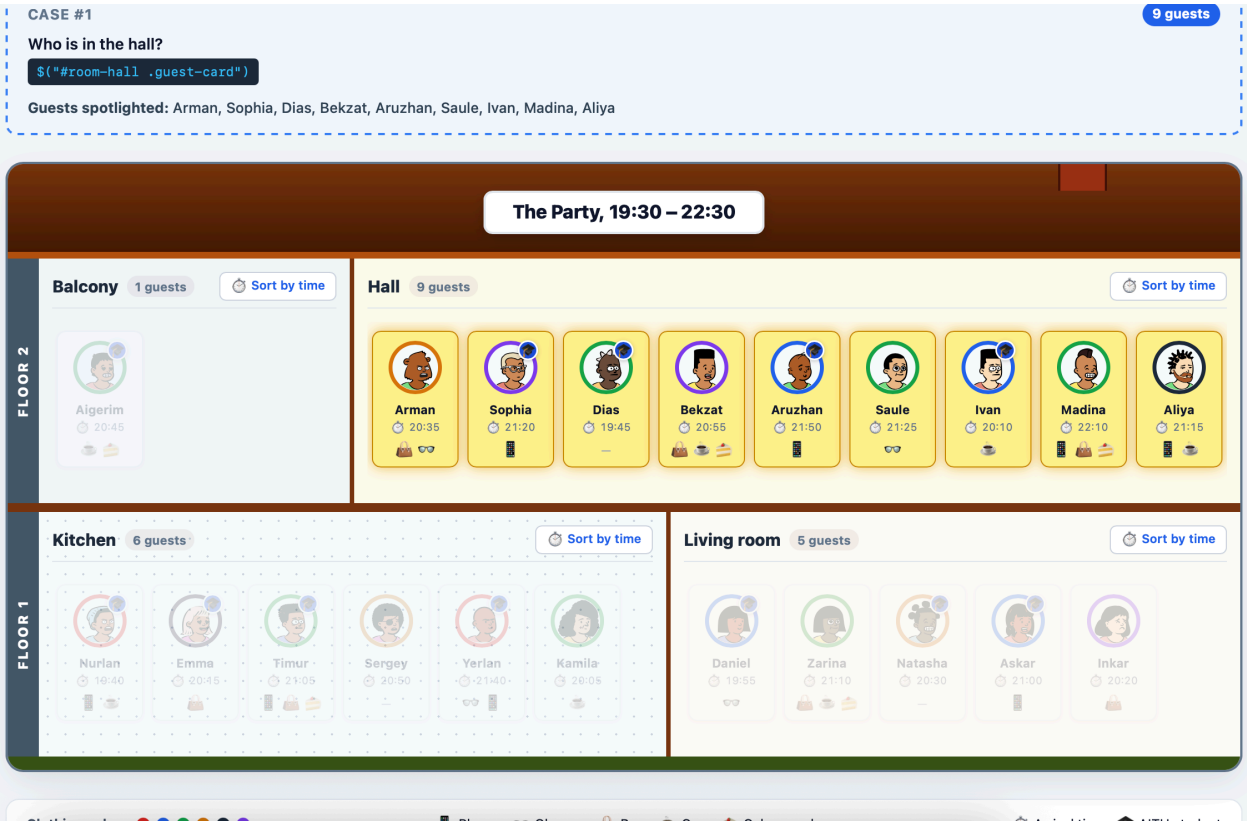


Figure: *Execution of Case 1 (Who is in the hall?) after Arman's transfer to Balcony. Hall result highlights 5 guests and no longer contains Arman.*

4.4. Effect 4 — Chronological Positional Sorting

Each room header contains a 'Sort by time' button. Clicking it reorders that room's cards chronologically by arrival time (earliest to latest). Clicking it a second time restores the original picture order. The animation uses a jQuery positional FLIP technique:

- **1.** First: Capture the starting offset() of every guest card in the room.
- **2.** Last: Reorder the cards in the DOM using jQuery append() and capture the new offset().
- **3.** Invert: Apply relative positional deltas (left = oldLeft - newLeft, top = oldTop - newTop) via .css() so cards visually remain at their old locations.
- **4.** Play: Animate left and top to 0 over 380ms via animate(), producing smooth sliding movement across positions without jumping.

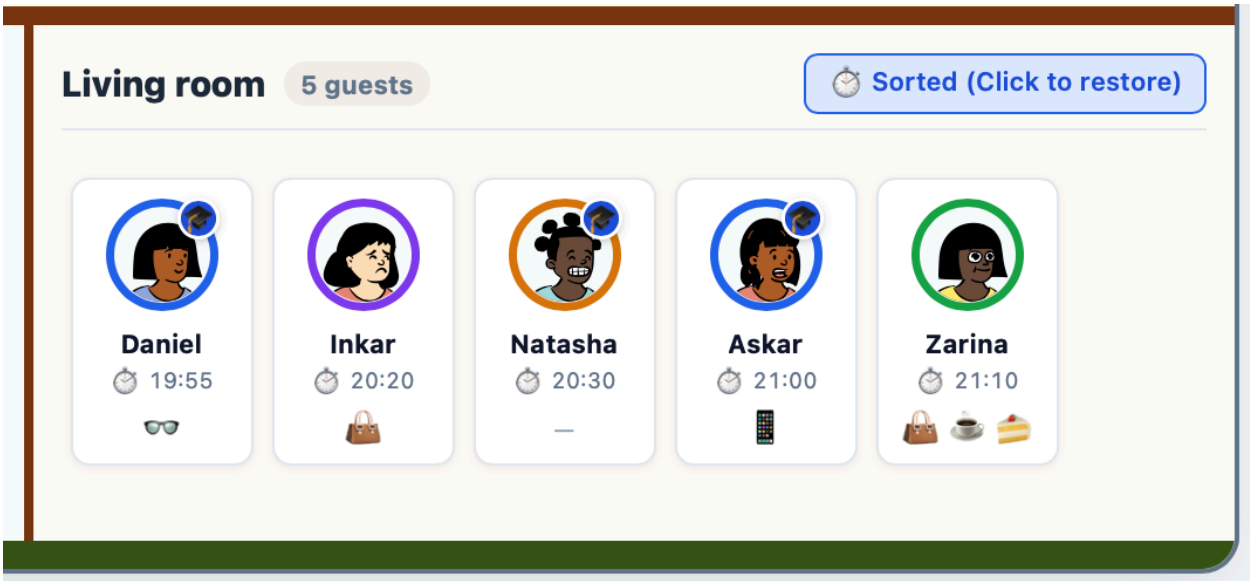


Figure: *Living Room after clicking 'Sort by time'. Guests slide smoothly into chronological order: Daniel (19:55), Inkar (20:20), Natasha (20:30), Askar (21:00), Zarina (21:10).*

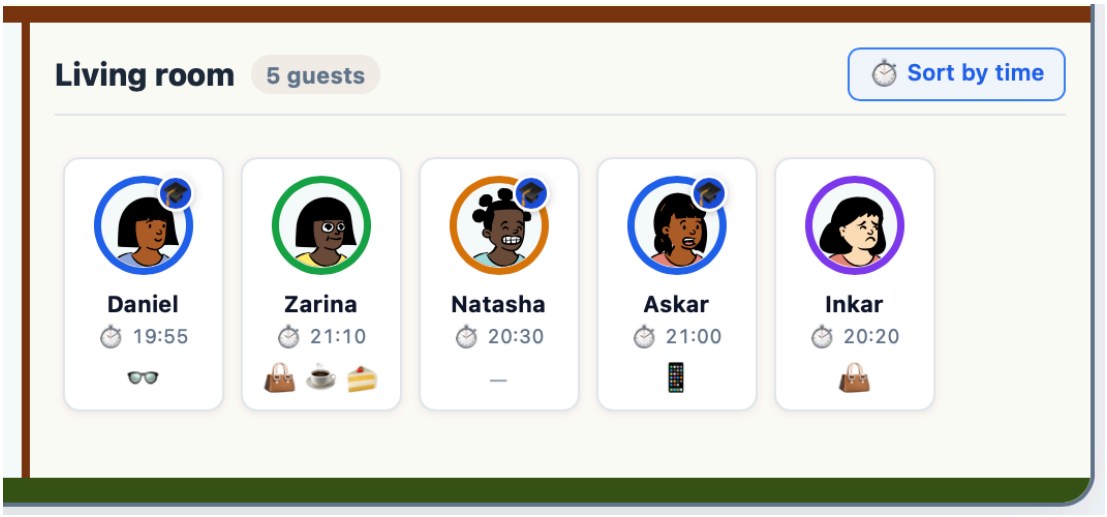


Figure: *Living Room after clicking 'Sort by time' a second time. Cards slide smoothly back to their original picture order: Daniel, Zarina, Natasha, Askar, Inkar.*

4.5. Effect 5 — Animation Queue & Fast-Click Parity

To prevent animation queue buildup, element duplication, and race conditions during rapid user interaction, strict queue management was implemented using jQuery's .stop(true, false), operation tokens, and parity tracking. When a user clicks the sort button five times in rapid succession, the state toggles an odd number of times (5), guaranteeing that the room cleanly settles in sorted order without shaking, jumping, or leaving residual animation queues.

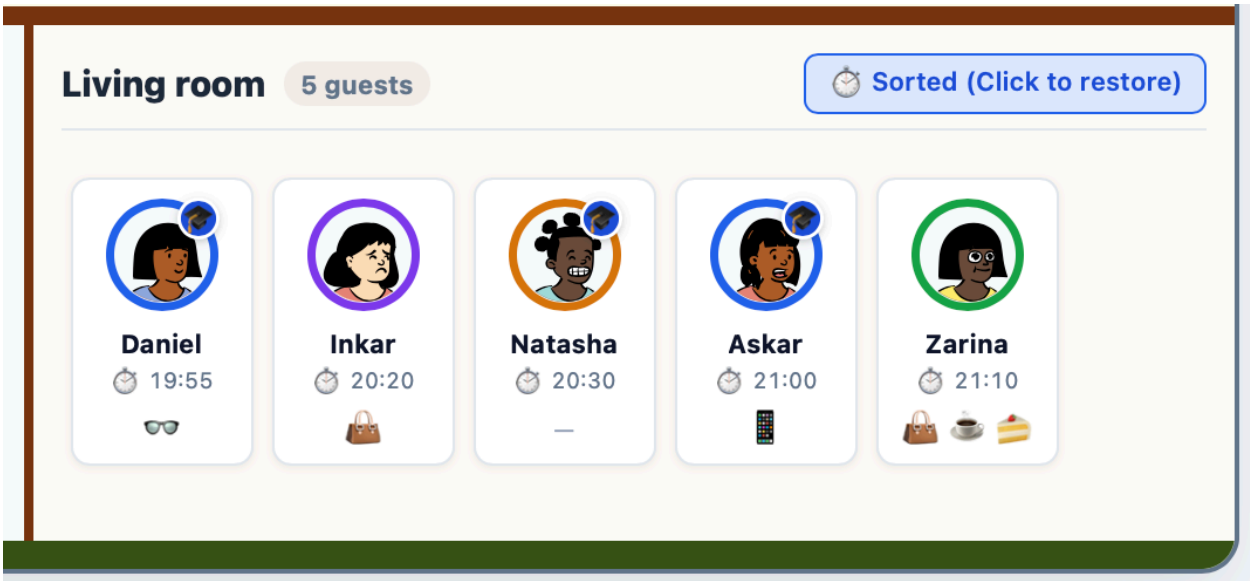


Figure: *Living Room after five rapid sorting clicks. The room settles cleanly in chronological sorted order with zero duplicate cards, stuck queues, or displaced elements.*

4.6. Authoritative Task 2 Verification Table

Table 2 documents the five authoritative acceptance tests specified in the assignment, comparing expected results against actual verified behavior:

Test Check	Procedure	Expected Result	Actual Verified Result	Status
Case 3, then Clear	Click Case 3 button, wait 350ms, then click Clear Spotlight.	Only AITU students on 2nd floor stay bright; Clear brings everybody back to full opacity.	Verified: Exactly 5 upstairs AITU students remained at opacity 1.0 while others faded to 0.2 over 300ms. Clear restored all 21 cards to opacity 1.0.	**Pass**

Click a guest, then Esc	Click any guest card (Aigerim), observe expansion, then press Escape.	The card grows out of the avatar with all data; Esc shrinks it back into the avatar.	Verified: Modal expanded smoothly from avatar offset (300ms) with full descriptive sentences; Escape key triggered 240ms shrink animation back into avatar.	**Pass**
Move Arman to balcony	Open Arman's card, click 'Move to Balcony', wait for animation, then execute Case 1.	Arman flies to end of balcony; hall closes the gap; Case 1 no longer finds him.	Verified: Arman animated across house (500ms), Hall gap closed (280ms), Balcony opened space. DOM & array updated; subsequent Case 1 query excludes Arman.	**Pass**
Sort living room twice	Click 'Sort by time' in Living room header, observe order, then click again.	Cards slide to 19:55, 20:20, 20:30, 21:00, 21:10, then slide back to picture order.	Verified: Cards smoothly slid into chronological order (19:55 → 20:20 → 20:30 → 21:00 → 21:10), second click restored Daniel → Zarina → Natasha → Askar → Inkar.	**Pass**
Sort 5 times fast	Click 'Sort by time' in Living room 5 times in rapid succession.	Cards end in sorted order without jumping or shaking; no animation queues repeat.	Verified: Handled odd-parity toggles cleanly. Finished in sorted order (19:55 to 21:10) without shaking, duplicate cards, or continuous animation queues.	**Pass**

5. Conclusion & Technical Insights

Completing Assignment #6 has significantly advanced my practical mastery of core frontend engineering, DOM manipulation, and animation sequencing using jQuery. By strictly avoiding external frameworks, CSS transitions, and CSS keyframe animations, I was required to solve complex motion challenges purely through algorithmic coordinate calculations and jQuery's effect APIs.

In Task 1, constructing the party house from a data model reinforced how data-* attributes and semantic class names bridge JavaScript data structures with DOM query engines. Implementing the 20 selector detective cases taught me the nuances between hierarchical descendant selectors, jQuery-specific extensions (:has(), :not()), structural position selectors (:last-child, :nth-child()), and traversal methods (.next(), .prev(), .nextAll(), .closest(), .add()). Solving Case 20 (the cake mystery) through a compound four-clue filter demonstrated the power of functional DOM filtering when analyzing complex relational conditions.

In Task 2, building the animated party interface revealed the critical importance of keeping application state synchronized across both the live DOM and memory arrays. Engineering the cross-house flying transfer required coordinating offset calculations, temporary clones, and simultaneous dimension animations so that moving elements feel physical rather than synthetic. Similarly, implementing chronological sorting via the FLIP technique demonstrated how measuring starting and ending geometry allows smooth positional sliding without layout jitter. Finally, handling rapid user clicks with .stop(true, false) and parity tracking provided essential lessons in concurrency control and animation queue hygiene, ensuring the interface remains deterministic, responsive, and robust under all conditions.